UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

INSTITUTO DE INFORMÁTICA

DISCIPLINA: Linguagens de Programação de Hardware - CI INOVADOR

Prof. Fernanda Lima Kastensmidt

2026

Trabalho - Individual

Projeto do Processador Neander em VERILOG

Aluno José Adriano Filho

Data de entrega: 28 de novembro de 2026

O computador NEANDER foi criado com intenções didáticas pelo prof. Raul Weber da UFRGS. (Livro: Arquitetura de Computadores Pessoais - Número 6, Raul Fernando Weber, Editora: Sangra Luzzatto)

O objetivo deste trabalho de SD é implementar o NEANDER usando a linguagem de descrição de hardware Verilog, simular esse circuito em um simulador lógico sem atraso, depois realizar a síntese lógica, mapeamento tecnológico, posicionamento e roteamento para um FPGA, realizar a simulação com atraso e ***prototipar o processador em uma placa de prototipação (*opcional)**

1) Deve-se inserir a instrução de Subtração (SUB) conforme os modos de operandos da instrução ADD. O SUB deve tratar dados em complemento de 2.

2) Programas a serem implementado no NEANDER na memória embarcada BRAM (descreva. coe para inicializar a BRAM)

1) Subtração de duas matrizes A e B 3x3 com dados de 8 bits, onde os dados das matrizes estão armazenados em memória
2) Multiplicação de dois valores A e B por soma sucessiva

1) Descrição do trabalho

Este trabalho apresenta a implementação, em Verilog, de um processador baseado na arquitetura didática Neander, desenvolvido no Vivado 2024.2 para o FPGA AMD Artix-7 XC7A100T-CSG324-1. O processador possui dados e endereços de 8 bits, memória de 256 palavras de 8 bits, acumulador (AC), contador de programa (PC), registrador de instrução (RI), registrador de endereço de memória (REM) e indicadores negativo (N) e zero (Z). A organização compreende um caminho de dados e uma unidade de controle baseada em máquina de estados.

Foi acrescentada a instrução SUB, com opcode 7h e endereçamento direto, seguindo o formato de ADD. A operação AC ← AC − M[endereço] produz um resultado de oito bits e atualiza N e Z. A aritmética é realizada módulo 256 e os valores negativos são interpretados em complemento de dois. O processador não possui indicador específico de overflow.

A memória foi implementada com o IP Block Memory Generator, como RAM de porta única inicializada por arquivo COE. Foram desenvolvidas duas aplicações: subtração elemento a elemento de matrizes 3 × 3 e multiplicação de valores positivos por somas sucessivas. Os testbenches verificaram os nove resultados das matrizes e, para a multiplicação de 7 por 5, o resultado 35, os valores parciais e o contador de repetições.

O projeto passou por simulação comportamental, síntese, posicionamento e roteamento. Os dois programas foram aprovados na simulação temporal após implementação. A subtração de matrizes atingiu HLT no ciclo 183 e a multiplicação no ciclo 344. A prototipação em placa era opcional e não foi realizada.

2) Verilog completo do Neander

As listagens a seguir apresentam os dez módulos RTL do processador. A memória memoria_neander é gerada pelo Block Memory Generator do Vivado, cuja configuração é descrita na seção 2.11. A seção 3 apresenta os testbenches e os arquivos COE utilizados para inicializar a BRAM.

2.1 neander.v

```
`timescale 1ns / 1ps

module neander (
    input  wire       clk,
    input  wire       reset,

    output wire       parado,
    output wire       erro,
    output wire [3:0] estado,

    // Saidas para acompanhar a execucao
    output wire [7:0] ac,
    output wire [7:0] pc,
    output wire [7:0] ri,
    output wire [7:0] rem,
    output wire [3:0] opcode,
    output wire       n,
    output wire       z,
    output wire [7:0] endereco_memoria,
    output wire [7:0] dado_memoria,
    output wire [7:0] resultado_ula
);

    // Ligacoes entre controle e caminho de dados
    wire carga_ac;
    wire carga_flags;
    wire carga_ri;
    wire carga_rem;
    wire carga_pc;
    wire incrementa_pc;

    wire [2:0] op_ula;
    wire seleciona_rem;
    wire habilita_memoria;
    wire escreve_memoria;

    unidade_controle u_controle (
        .clk(clk),
        .reset(reset),
        .opcode(opcode),
        .n(n),
        .z(z),

        .carga_ac(carga_ac),
        .carga_flags(carga_flags),
        .carga_ri(carga_ri),
        .carga_rem(carga_rem),
        .carga_pc(carga_pc),
        .incrementa_pc(incrementa_pc),

        .op_ula(op_ula),
        .seleciona_rem(seleciona_rem),
        .habilita_memoria(habilita_memoria),
        .escreve_memoria(escreve_memoria),

        .parado(parado),
        .erro(erro),
        .estado(estado)
    );

    caminho_dados u_dados (
        .clk(clk),
```

```
        .reset(reset),

        .carga_ac(carga_ac),
        .carga_flags(carga_flags),
        .carga_ri(carga_ri),
        .carga_rem(carga_rem),
        .carga_pc(carga_pc),
        .incrementa_pc(incrementa_pc),

        .op_ula(op_ula),
        .seleciona_rem(seleciona_rem),
        .habilita_memoria(habilita_memoria),
        .escreve_memoria(escreve_memoria),

        .ac(ac),
        .pc(pc),
        .ri(ri),
        .rem(rem),
        .opcode(opcode),
        .n(n),
        .z(z),
        .endereco_memoria(endereco_memoria),
        .dado_memoria(dado_memoria),
        .resultado_ula(resultado_ula)
    );

endmodule
```

2.2 unidade_controle.v

```
`timescale 1ns / 1ps

module unidade_controle (
    input  wire       clk,
    input  wire       reset,
    input  wire [3:0] opcode,
    input  wire       n,
    input  wire       z,

    output reg        carga_ac,
    output reg        carga_flags,
    output reg        carga_ri,
    output reg        carga_rem,
    output reg        carga_pc,
    output reg        incrementa_pc,

    output reg  [2:0] op_ula,
    output reg        seleciona_rem,
    output reg        habilita_memoria,
    output reg        escreve_memoria,

    output reg        parado,
    output reg        erro,
    output reg  [3:0] estado
);

    // Codigos das instrucoes: quatro bits superiores do RI
    localparam OP_NOP = 4'h0,
               OP_STA = 4'h1,
               OP_LDA = 4'h2,
               OP_ADD = 4'h3,
               OP_OR  = 4'h4,
               OP_AND = 4'h5,
               OP_NOT = 4'h6,
               OP_SUB = 4'h7,
               OP_JMP = 4'h8,
               OP_JN  = 4'h9,
               OP_JZ  = 4'hA,
               OP_HLT = 4'hF;

    // Estados da unidade de controle
```

```
localparam BUSCA_INSTRUCAO   = 4'd0,
           CAPTURA_INSTRUCAO = 4'd1,
           DECODIFICA        = 4'd2,
           BUSCA_ENDERECO    = 4'd3,
           CAPTURA_ENDERECO  = 4'd4,
           LE_OPERANDO       = 4'd5,
           EXECUTA_ULA       = 4'd6,
           ESCREVE_MEMORIA   = 4'd7,
           DESVIO            = 4'd8,
           EXECUTA_NOT       = 4'd9,
           PARADO            = 4'd10,
           ERRO              = 4'd11;

reg [3:0] proximo_estado;

// 1. Registrador de estado, com reset sincrono
always @(posedge clk) begin
    if (reset)
        estado <= BUSCA_INSTRUCAO;
    else
        estado <= proximo_estado;
end

// 2. Logica de transicao entre estados
always @(*) begin
    proximo_estado = ERRO;

    case (estado)
        BUSCA_INSTRUCAO:
            proximo_estado = CAPTURA_INSTRUCAO;

        CAPTURA_INSTRUCAO:
            proximo_estado = DECODIFICA;

        DECODIFICA: begin
            case (opcode)
                OP_NOP:
                    proximo_estado = BUSCA_INSTRUCAO;

                OP_NOT:
                    proximo_estado = EXECUTA_NOT;

                OP_HLT:
                    proximo_estado = PARADO;

                OP_STA, OP_LDA, OP_ADD, OP_OR,
                OP_AND, OP_SUB, OP_JMP, OP_JN, OP_JZ:
                    proximo_estado = BUSCA_ENDERECO;

                default:
                    proximo_estado = ERRO;
            endcase
        end

        BUSCA_ENDERECO:
            proximo_estado = CAPTURA_ENDERECO;

        CAPTURA_ENDERECO: begin
            case (opcode)
                OP_LDA, OP_ADD, OP_OR, OP_AND, OP_SUB:
                    proximo_estado = LE_OPERANDO;

                OP_STA:
                    proximo_estado = ESCREVE_MEMORIA;

                OP_JMP, OP_JN, OP_JZ:
                    proximo_estado = DESVIO;

                default:
                    proximo_estado = ERRO;
            endcase
        end
```

```
        LE_OPERANDO:
            proximo_estado = EXECUTA_ULA;

        EXECUTA_ULA, ESCREVE_MEMORIA,
        DESVIO, EXECUTA_NOT:
            proximo_estado = BUSCA_INSTRUCAO;

        PARADO:
            proximo_estado = PARADO;

        ERRO:
            proximo_estado = ERRO;

        default:
            proximo_estado = ERRO;
    endcase
end

// 3. Geracao dos sinais de controle
always @(*) begin
    // Valores padrao: nenhuma alteracao ou acesso
    carga_ac          = 1'b0;
    carga_flags       = 1'b0;
    carga_ri          = 1'b0;
    carga_rem         = 1'b0;
    carga_pc          = 1'b0;
    incrementa_pc     = 1'b0;

    op_ula            = 3'b000;
    seleciona_rem     = 1'b0;
    habilita_memoria  = 1'b0;
    escreve_memoria   = 1'b0;

    parado            = 1'b0;
    erro              = 1'b0;

    // Mantem a operacao selecionada enquanto o RI nao muda.
    case (opcode)
        OP_LDA: op_ula = 3'b000;
        OP_ADD: op_ula = 3'b001;
        OP_SUB: op_ula = 3'b010;
        OP_AND: op_ula = 3'b011;
        OP_OR:  op_ula = 3'b100;
        OP_NOT: op_ula = 3'b101;
        default: op_ula = 3'b000;
    endcase

    // Mantem os controles inativos durante o reset
    if (!reset) begin
        case (estado)
            BUSCA_INSTRUCAO: begin
                habilita_memoria = 1'b1;
            end

            CAPTURA_INSTRUCAO: begin
                carga_ri      = 1'b1;
                incrementa_pc = 1'b1;
            end

            BUSCA_ENDERECO: begin
                habilita_memoria = 1'b1;
            end

            CAPTURA_ENDERECO: begin
                carga_rem     = 1'b1;
                incrementa_pc = 1'b1;
            end

            LE_OPERANDO: begin
                seleciona_rem    = 1'b1;
                habilita_memoria = 1'b1;
```

```
                end

                EXECUTA_ULA: begin
                    // Atualiza somente para operacoes validas
                    case (opcode)
                        OP_LDA, OP_ADD, OP_SUB, OP_AND, OP_OR: begin
                            carga_ac    = 1'b1;
                            carga_flags = 1'b1;
                        end
                        default: begin
                            carga_ac    = 1'b0;
                            carga_flags = 1'b0;
                        end
                    endcase
                end

                ESCREVE_MEMORIA: begin
                    seleciona_rem    = 1'b1;
                    habilita_memoria = 1'b1;
                    escreve_memoria  = 1'b1;
                end

                DESVIO: begin
                    case (opcode)
                        OP_JMP: carga_pc = 1'b1;
                        OP_JN:  carga_pc = n;
                        OP_JZ:  carga_pc = z;
                        default: carga_pc = 1'b0;
                    endcase
                end

                EXECUTA_NOT: begin
                    carga_ac    = 1'b1;
                    carga_flags = 1'b1;
                end

                PARADO: begin
                    parado = 1'b1;
                end

                ERRO: begin
                    parado = 1'b1;
                    erro   = 1'b1;
                end

                default: begin
                    // Demais estados mantem os valores padrao
                end
            endcase
        end
    end

endmodule
```

2.3 caminho_dados.v

```
`timescale 1ns / 1ps

module caminho_dados (
    input  wire       clk,
    input  wire       reset,

    // Controles dos registradores
    input  wire       carga_ac,
    input  wire       carga_flags,
    input  wire       carga_ri,
    input  wire       carga_rem,
    input  wire       carga_pc,
    input  wire       incrementa_pc,

    // Controles da ULA e da memoria
```

```
    input  wire [2:0] op_ula,
    input  wire       seleciona_rem,
    input  wire       habilita_memoria,
    input  wire       escreve_memoria,

    // Saidas para controle e acompanhamento
    output wire [7:0] ac,
    output wire [7:0] pc,
    output wire [7:0] ri,
    output wire [7:0] rem,
    output wire [3:0] opcode,
    output wire       n,
    output wire       z,
    output wire [7:0] endereco_memoria,
    output wire [7:0] dado_memoria,
    output wire [7:0] resultado_ula
);

    wire n_ula;
    wire z_ula;

    wire memoria_ativa;
    wire escrita_ativa;

    // Impede acessos durante o reset do processador
    assign memoria_ativa = habilita_memoria & ~reset;
    assign escrita_ativa = escreve_memoria & ~reset;

    contador_programa u_pc (
        .clk(clk),
        .reset(reset),
        .carga(carga_pc),
        .incrementa(incrementa_pc),
        .endereco(rem),
        .pc(pc)
    );

    registrador_instrucao u_ri (
        .clk(clk),
        .reset(reset),
        .carga(carga_ri),
        .dados(dado_memoria),
        .ri(ri),
        .opcode(opcode)
    );

    registrador_endereco u_rem (
        .clk(clk),
        .reset(reset),
        .carga(carga_rem),
        .dados(dado_memoria),
        .rem(rem)
    );

    seletor_endereco u_seletor (
        .pc(pc),
        .rem(rem),
        .seleciona_rem(seleciona_rem),
        .endereco(endereco_memoria)
    );

    memoria_neander u_memoria (
        .clka(clk),
        .ena(memoria_ativa),
        .wea(escrita_ativa),
        .addra(endereco_memoria),
        .dina(ac),
        .douta(dado_memoria)
    );

    ula u_ula (
        .a(ac),
```

```
        .b(dado_memoria),
        .op(op_ula),
        .resultado(resultado_ula),
        .n(n_ula),
        .z(z_ula)
    );

    acumulador u_ac (
        .clk(clk),
        .reset(reset),
        .carga(carga_ac),
        .dados(resultado_ula),
        .q(ac)
    );

    assign z = (ac == 8'h00);
    registrador_flags u_flags (
        .clk(clk),
        .reset(reset),
        .carga(carga_flags),
        .n_in(n_ula),
        .z_in(z_ula),
        .n(n),
        .z()
    );

endmodule
```

2.4 ula.v

```
`timescale 1ns / 1ps

module ula (
    input  wire [7:0] a,
    input  wire [7:0] b,
    input  wire [2:0] op,

    output reg  [7:0] resultado,
    output wire       n,
    output wire       z
);

    always @(*) begin
        case (op)
            3'b000: resultado = b;       // LDA
            3'b001: resultado = a + b;   // ADD
            3'b010: resultado = a - b;   // SUB
            3'b011: resultado = a & b;   // AND
            3'b100: resultado = a | b;   // OR
            3'b101: resultado = ~a;      // NOT
            default: resultado = 8'b00000000;
        endcase
    end

    assign n = resultado[7];
    assign z = (resultado == 8'b00000000);

endmodule
```

2.5 acumulador.v

```
`timescale 1ns / 1ps

module acumulador (
    input  wire       clk,
    input  wire       reset,
    input  wire       carga,
    input  wire [7:0] dados,

    output reg  [7:0] q
);
```

```
    always @(posedge clk) begin
        if (reset)
            q <= 8'd0;
        else if (carga)
            q <= dados;
    end

endmodule
```

2.6 contador_programa.v

```
`timescale 1ns / 1ps

module contador_programa (
    input  wire       clk,
    input  wire       reset,
    input  wire       carga,
    input  wire       incrementa,
    input  wire [7:0] endereco,

    output reg  [7:0] pc
);

    always @(posedge clk) begin
        if (reset)
            pc <= 8'd0;
        else if (carga)
            pc <= endereco;
        else if (incrementa)
            pc <= pc + 8'd1;
    end

endmodule
```

2.7 registrador_instrucao.v

```
`timescale 1ns / 1ps

module registrador_instrucao (
    input  wire       clk,
    input  wire       reset,
    input  wire       carga,
    input  wire [7:0] dados,

    output reg  [7:0] ri,
    output wire [3:0] opcode
);

    always @(posedge clk) begin
        if (reset)
            ri <= 8'h00;
        else if (carga)
            ri <= dados;
    end

    assign opcode = ri[7:4];

endmodule
```

2.8 registrador_endereco.v

```
`timescale 1ns / 1ps

module registrador_endereco (
    input  wire       clk,
    input  wire       reset,
    input  wire       carga,
    input  wire [7:0] dados,

    output reg  [7:0] rem
```

```
);

    always @(posedge clk) begin
        if (reset)
            rem <= 8'd0;
        else if (carga)
            rem <= dados;
    end

endmodule
```

2.9 registrador_flags.v

```
`timescale 1ns / 1ps

module registrador_flags (
    input  wire clk,
    input  wire reset,
    input  wire carga,
    input  wire n_in,
    input  wire z_in,

    output reg  n,
    output reg  z
);

    always @(posedge clk) begin
        if (reset) begin
            n <= 1'b0;
            z <= 1'b1;
        end
        else if (carga) begin
            n <= n_in;
            z <= z_in;
        end
    end

endmodule
```

2.10 seletor_endereco.v

```
`timescale 1ns / 1ps

module seletor_endereco (
    input  wire [7:0] pc,
    input  wire [7:0] rem,
    input  wire       seleciona_rem,

    output wire [7:0] endereco
);

    assign endereco = seleciona_rem ? rem : pc;

endmodule
```

2.11 Memória gerada pelo Vivado

O caminho de dados instancia memoria_neander com clka, ena, wea, addra[7:0], dina[7:0] e douta[7:0]. A memória foi configurada no Block Memory Generator 8.4 com interface Native, Single Port RAM, largura de leitura e escrita de oito bits, profundidade de 256 palavras, pino ENA e modo Write First. Os registradores opcionais de saída estão desabilitados, com latência de leitura de um ciclo. A inicialização utiliza o arquivo COE da aplicação e preenche as demais posições com zero.

3) Testbench completo

As seções 3.1 e 3.2 apresentam os testes comportamentais dos dois programas. A seção 3.3 contém o teste funcional da netlist implementada. Os testes temporais finais estão nas seções 3.5 e 3.6. Essas listagens verificam os resultados, a contagem de ciclos e a estabilidade após HLT.

3.1 Testbench da subtração de matrizes

O teste aplica clock de 50 MHz e verifica nove transações de escrita nos endereços A0h a A8h, os valores de AC e as flags N e Z. Também confere 28 instruções, 183 ciclos, o estado final e a manutenção da parada após HLT. A conferência acompanha o endereço e o dado nas transações de escrita. O teste não faz uma leitura independente de todas as posições de C após HLT.

```
`timescale 1ns / 1ps

module neander_matrizes_tb;

    reg clk;
    reg reset;

    wire parado;
    wire erro;
    wire [3:0] estado;
    wire [7:0] ac, pc, ri, rem;
    wire [3:0] opcode;
    wire n, z;
    wire [7:0] endereco_memoria;
    wire [7:0] dado_memoria;
    wire [7:0] resultado_ula;

    integer ciclos;
    integer instrucoes;
    integer escritas;

    reg resultados_corretos;

    // Resultados esperados, na ordem das escritas
    reg [7:0] esperado [0:8];

    neander dut (
        .clk(clk),
        .reset(reset),
        .parado(parado),
        .erro(erro),
        .estado(estado),
        .ac(ac),
        .pc(pc),
        .ri(ri),
        .rem(rem),
        .opcode(opcode),
        .n(n),
        .z(z),
        .endereco_memoria(endereco_memoria),
        .dado_memoria(dado_memoria),
        .resultado_ula(resultado_ula)
    );

    // Clock de 50 MHz
    initial begin
        clk = 1'b0;
        forever #10 clk = ~clk;
    end

    // Conta instrucoes e verifica escritas
    always @(posedge clk) begin
```

```
    if (reset) begin
        instrucoes          <= 0;
        escritas            <= 0;
        resultados_corretos <= 1'b1;
    end
    else begin
        // A borda que encerra DECODIFICA conta a instrucao
        if (estado == 4'd2)
            instrucoes <= instrucoes + 1;

        if (dut.habilita_memoria &&
            dut.escreve_memoria) begin

            escritas <= escritas + 1;

            // Evita acessar o vetor fora dos limites
            if (escritas < 9) begin
                if (endereco_memoria !== (8'hA0 + escritas) ||
                    ac !== esperado[escritas] ||
                    n !== esperado[escritas][7] ||
                    z !== (esperado[escritas] == 8'h00)) begin

                    resultados_corretos <= 1'b0;

                    $display(
                        "ERRO na escrita %0d: endereco=%h dado=%h esperado=%h",
                        escritas, endereco_memoria,
                        ac, esperado[escritas]
                    );
                end

                $display(
                    "C[%0d][%0d]: endereco=%h resultado=%0d hex=%h N=%b Z=%b",
                    escritas / 3, escritas % 3,
                    endereco_memoria, $signed(ac), ac, n, z
                );
            end
            else begin
                resultados_corretos <= 1'b0;
                $display("ERRO: escrita adicional inesperada.");
            end
        end
    end
end

initial begin
    esperado[0] = 8'h08;
    esperado[1] = 8'hFE;
    esperado[2] = 8'h00;
    esperado[3] = 8'hFE;
    esperado[4] = 8'h08;
    esperado[5] = 8'h06;
    esperado[6] = 8'h06;
    esperado[7] = 8'hFB;
    esperado[8] = 8'h00;

    reset = 1'b1;
    ciclos = 0;

    @(negedge clk);             // 20 ns
    reset = 1'b0;

    // Limite para detectar execucao que nao termina
    while (parado !== 1'b1 && ciclos < 300) begin
        @(negedge clk);
        ciclos = ciclos + 1;
    end
```

```
        if (parado !== 1'b1) begin
            $display("ERRO: tempo limite sem parada.");
            $stop;
        end
        else if (erro !== 1'b0) begin
            $display("ERRO: opcode invalido.");
            $stop;
        end
        else if (resultados_corretos !== 1'b1 ||
                 escritas != 9 ||
                 instrucoes != 28 ||
                 ciclos != 183 ||
                 ac !== 8'h00 ||
                 pc !== 8'h37 ||
                 ri !== 8'hF0 ||
                 rem !== 8'hA8 ||
                 n !== 1'b0 ||
                 z !== 1'b1 ||
                 estado !== 4'd10) begin

            $display("ERRO: verificacao final.");
            $display("Instrucoes=%0d ciclos=%0d escritas=%0d",
                     instrucoes, ciclos, escritas);
            $display("AC=%h PC=%h RI=%h REM=%h N=%b Z=%b",
                     ac, pc, ri, rem, n, z);
            $stop;
        end
        else begin
            $display("SUCESSO: nove resultados corretos.");
            $display("Instrucoes executadas: %0d", instrucoes);
            $display("Ciclos de execucao: %0d", ciclos);
            $display("Tempo a 50 MHz: %0d ns", ciclos * 20);

            repeat (3) @(negedge clk);

            if (estado !== 4'd10 ||
                pc !== 8'h37 ||
                ac !== 8'h00 ||
                n !== 1'b0 ||
                z !== 1'b1 ||
                dut.habilita_memoria !== 1'b0) begin

                $display("ERRO: comportamento apos HLT.");
                $stop;
            end
            else begin
                $display("HLT manteve o processador parado.");
                $finish;
            end
        end
    end

endmodule
```

3.2 Testbench da multiplicação por somas sucessivas

O teste confere 12 transações de escrita: inicialização do resultado e do contador, seguida de cinco pares de atualizações. Os resultados parciais esperados são 7, 14, 21, 28 e 35, com contador de 4 até 0. A verificação final exige 53 instruções, 344 ciclos, AC = 23h e PC = 1Dh, além da permanência no estado de parada.

```
`timescale 1ns / 1ps

module neander_multiplicacao_tb;
```

```
reg clk;
reg reset;

wire parado, erro;
wire [3:0] estado, opcode;
wire [7:0] ac, pc, ri, rem;
wire n, z;
wire [7:0] endereco_memoria;
wire [7:0] dado_memoria;
wire [7:0] resultado_ula;

integer ciclos;
integer instrucoes;
integer escritas;

reg resultados_corretos;

reg [7:0] enderecos_esperados [0:11];
reg [7:0] dados_esperados     [0:11];

integer i;

neander dut (
    .clk(clk),
    .reset(reset),
    .parado(parado),
    .erro(erro),
    .estado(estado),
    .ac(ac),
    .pc(pc),
    .ri(ri),
    .rem(rem),
    .opcode(opcode),
    .n(n),
    .z(z),
    .endereco_memoria(endereco_memoria),
    .dado_memoria(dado_memoria),
    .resultado_ula(resultado_ula)
);

initial begin
    clk = 1'b0;
    forever #10 clk = ~clk;
end

// Conta instrucoes e confere cada escrita
always @(posedge clk) begin
    if (reset) begin
        instrucoes          <= 0;
        escritas            <= 0;
        resultados_corretos <= 1'b1;
    end
    else begin
        if (estado == 4'd2)
            instrucoes <= instrucoes + 1;

        if (dut.habilita_memoria &&
            dut.escreve_memoria) begin

            escritas <= escritas + 1;

            if (escritas < 12) begin
                if (endereco_memoria !==
                    enderecos_esperados[escritas] ||
                    ac !== dados_esperados[escritas]) begin

                    resultados_corretos <= 1'b0;
```

```
                    $display(
                        "ERRO na escrita %0d: endereco=%h dado=%h",
                        escritas, endereco_memoria, ac
                    );
                end

                if (endereco_memoria == 8'h82)
                    $display("Resultado parcial: %0d", ac);
                else if (endereco_memoria == 8'h83)
                    $display("Contador: %0d", ac);
            end
            else begin
                resultados_corretos <= 1'b0;
                $display("ERRO: escrita adicional inesperada.");
            end
        end
    end
end

initial begin
    // Escritas de inicializacao
    enderecos_esperados[0] = 8'h82;
    dados_esperados[0]     = 8'd0;

    enderecos_esperados[1] = 8'h83;
    dados_esperados[1]     = 8'd5;

    // Cada repeticao escreve resultado e contador
    for (i = 0; i < 5; i = i + 1) begin
        enderecos_esperados[2 + 2*i] = 8'h82;
        dados_esperados[2 + 2*i]     = 7 * (i + 1);

        enderecos_esperados[3 + 2*i] = 8'h83;
        dados_esperados[3 + 2*i]     = 4 - i;
    end

    reset = 1'b1;
    ciclos = 0;

    @(negedge clk);             // 20 ns
    reset = 1'b0;

    while (parado !== 1'b1 && ciclos < 500) begin
        @(negedge clk);
        ciclos = ciclos + 1;
    end

    if (parado !== 1'b1) begin
        $display("ERRO: tempo limite sem parada.");
        $stop;
    end
    else if (erro !== 1'b0) begin
        $display("ERRO: opcode invalido.");
        $stop;
    end
    else if (resultados_corretos !== 1'b1 ||
             escritas != 12 ||
             instrucoes != 53 ||
             ciclos != 344 ||
             ac !== 8'h23 ||
             pc !== 8'h1D ||
             ri !== 8'hF0 ||
             rem !== 8'h82 ||
             n !== 1'b0 ||
             z !== 1'b0 ||
             estado !== 4'd10) begin
```

```
            $display("ERRO: verificacao final.");
            $display("Instrucoes=%0d ciclos=%0d escritas=%0d",
                     instrucoes, ciclos, escritas);
            $display("AC=%h PC=%h RI=%h REM=%h N=%b Z=%b",
                     ac, pc, ri, rem, n, z);
            $stop;
        end
        else begin
            $display("SUCESSO: 7 x 5 = %0d.", ac);
            $display("Somas parciais e contador corretos.");
            $display("Instrucoes executadas: %0d", instrucoes);
            $display("Ciclos: %0d", ciclos);
            $display("Tempo a 50 MHz: %0d ns", ciclos * 20);

            repeat (3) @(negedge clk);

            if (estado !== 4'd10 ||
                ac !== 8'h23 ||
                pc !== 8'h1D ||
                dut.habilita_memoria !== 1'b0) begin

                $display("ERRO: comportamento apos HLT.");
                $stop;
            end
            else begin
                $display("HLT manteve o processador parado.");
                $finish;
            end
        end
    end

endmodule
```

3.3 Testbench da simulação funcional após implementação

Este teste observa as portas do processador, mantém reset por 200 ns e verifica as saídas 1 ns após a borda de descida. Foi aprovado na simulação funcional da netlist implementada, sem anotação dos atrasos de propagação. Os ensaios temporais aprovados utilizam os testbenches finais das seções 3.5 e 3.6.

```
`timescale 1ns / 1ps

module neander_multiplicacao_timing_tb;

    reg clk;
    reg reset;

    wire parado, erro;
    wire [3:0] estado, opcode;
    wire [7:0] ac, pc, ri, rem;
    wire n, z;
    wire [7:0] endereco_memoria;
    wire [7:0] dado_memoria;
    wire [7:0] resultado_ula;

    integer ciclos;

    neander dut (
        .clk(clk),
        .reset(reset),
        .parado(parado),
        .erro(erro),
        .estado(estado),
        .ac(ac),
        .pc(pc),
        .ri(ri),
        .rem(rem),
```

```
    .opcode(opcode),
    .n(n),
    .z(z),
    .endereco_memoria(endereco_memoria),
    .dado_memoria(dado_memoria),
    .resultado_ula(resultado_ula)
);

// Clock de 50 MHz
initial begin
    clk = 1'b0;
    forever #10 clk = ~clk;
end

initial begin
    reset = 1'b1;
    ciclos = 0;

    // Aguarda a inicializacao global do modelo FPGA
    // e varias bordas com reset ativo.
    repeat (10) @(negedge clk); // 200 ns
    reset = 1'b0;

    // Observa as saidas 1 ns depois da borda de descida:
    // 11 ns depois da ultima borda de subida.
    while (parado !== 1'b1 && ciclos < 500) begin
        @(negedge clk);
        #1;
        ciclos = ciclos + 1;
    end

    if (parado !== 1'b1) begin
        $display("ERRO: tempo limite sem parada.");
        $stop;
    end
    else if (erro !== 1'b0) begin
        $display("ERRO: processador sinalizou falha.");
        $stop;
    end
    else if (ac !== 8'h23 ||
             pc !== 8'h1D ||
             ri !== 8'hF0 ||
             rem !== 8'h82 ||
             n !== 1'b0 ||
             z !== 1'b0 ||
             ciclos != 344) begin

        $display("ERRO: resultado inesperado.");
        $display("AC=%h PC=%h RI=%h REM=%h N=%b Z=%b",
                 ac, pc, ri, rem, n, z);
        $display("Ciclos observados: %0d", ciclos);
        $stop;
    end
    else begin
        $display("SUCESSO: multiplicacao com atraso.");
        $display("7 x 5 = %0d", ac);
        $display("Ciclos: %0d", ciclos);
        $display("Tempo em ciclos a 50 MHz: %0d ns",
                 ciclos * 20);

        repeat (3) @(negedge clk);
        #1;

        if (parado !== 1'b1 ||
            erro !== 1'b0 ||
            ac !== 8'h23 ||
            pc !== 8'h1D ||
```

```
                ri !== 8'hF0 ||
                rem !== 8'h82 ||
                n !== 1'b0 ||
                z !== 1'b0) begin

                $display("ERRO: valores mudaram apos HLT.");
                $stop;
            end
            else begin
                $display("HLT manteve os valores finais.");
                $finish;
            end
        end
    end

endmodule
```

3.4 Arquivos COE das aplicações

Os valores são hexadecimais. Cada entrada do vetor inicializa uma palavra de 8 bits, em ordem crescente de endereço a partir de 00h. Na configuração do IP, as posições não especificadas são preenchidas com zero.

programa_matrizes.coe

```
memory_initialization_radix=16;
memory_initialization_vector=
20,80,70,90,10,A0,20,81,
70,91,10,A1,20,82,70,92,
10,A2,20,83,70,93,10,A3,
20,84,70,94,10,A4,20,85,
70,95,10,A5,20,86,70,96,
10,A6,20,87,70,97,10,A7,
20,88,70,98,10,A8,F0,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
0A,05,08,03,0C,07,09,04,
06,00,00,00,00,00,00,00,
02,07,08,05,04,01,03,09,
06,00,00,00,00,00,00,00,
55,55,55,55,55,55,55,55,
55;
```

A matriz A ocupa 80h a 88h, B ocupa 90h a 98h e C ocupa A0h a A8h. As posições de C iniciam com 55h. O programa contém nove sequências LDA, SUB e STA e termina com HLT.

programa_multiplicacao.coe

```
memory_initialization_radix=16;
memory_initialization_vector=
20,84,10,82,20,81,10,83,
20,83,A0,1A,20,82,30,80,
10,82,20,83,70,85,10,83,
80,08,20,82,F0,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
```

```
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
07,05,00,00,00,01;
```

Os dados em 80h a 85h são, respectivamente, multiplicando 7, multiplicador 5, resultado, contador, constante zero e constante um. O programa testa o contador, acumula o multiplicando e decrementa o contador com SUB até atingir zero.

3.5 Testbench final da multiplicação com atraso

A listagem a seguir corresponde ao neander_multiplicacao_diag_tb usado no ensaio temporal aprovado. O teste confere 53 pontos de verificação, observa HLT no ciclo 344 e verifica os ciclos 345 a 347. A amostragem ocorre 18 ns após a subida externa do clock.

```
`timescale 1ns / 1ps

module neander_multiplicacao_diag_tb;

    reg clk;
    reg reset;

    wire parado, erro;
    wire [3:0] estado, opcode;
    wire [7:0] ac, pc, ri, rem;
    wire n, z;
    wire [7:0] endereco_memoria;
    wire [7:0] dado_memoria;
    wire [7:0] resultado_ula;

    integer ciclos;

    neander dut (
        .clk(clk),
        .reset(reset),
        .parado(parado),
        .erro(erro),
        .estado(estado),
        .ac(ac),
        .pc(pc),
        .ri(ri),
        .rem(rem),
        .opcode(opcode),
        .n(n),
        .z(z),
        .endereco_memoria(endereco_memoria),
        .dado_memoria(dado_memoria),
        .resultado_ula(resultado_ula)
    );

    // Referencia da imagem multiplicacao_hidra.coe. So observa portas do top.
    initial begin
        clk = 0;
        forever #10 clk = ~clk;
    end

    task conferir;
        input [7:0] pc_ref, ri_ref, rem_ref, ac_ref;
        input n_ref, z_ref;
        begin
            if ({estado, pc, ri, rem, ac, n, z, parado, erro} !==
                {4'd2, pc_ref, ri_ref, rem_ref, ac_ref, n_ref, z_ref, 2'b00}) begin
                $display("DIVERGENCIA ciclo=%0d PC_instrucao=%h", ciclos, pc_ref-8'd1);
                $display("ESPERADO estado=2 PC=%h RI=%h REM=%h AC=%h N=%b Z=%b parado=0 erro=0",
                         pc_ref, ri_ref, rem_ref, ac_ref, n_ref, z_ref);
                $display("OBSERVADO estado=%h PC=%h RI=%h REM=%h AC=%h N=%b Z=%b parado=%b erro=%b",
                         estado, pc, ri, rem, ac, n, z, parado, erro);
```

```
            $finish;
        end
    end
endtask

initial begin
    reset = 1;
    repeat (10) @(negedge clk);
    reset = 0;
    for (ciclos = 1; ciclos <= 347; ciclos = ciclos + 1) begin
        @(negedge clk);
        #8; // 18 ns apos subida externa, 2 ns antes da proxima subida
        $display("TRACE c=%0d estado=%h PC=%h RI=%h AC=%h N=%b Z=%b REM=%h endereco=%h dado=%h",
                 ciclos, estado, pc, ri, ac, n, z, rem, endereco_memoria, dado_memoria);
        case (ciclos)
            2: conferir(8'h01, 8'h20, 8'h00, 8'h00, 1'b0, 1'b1); // instrucao 1, endereco 00
            9: conferir(8'h03, 8'h10, 8'h84, 8'h00, 1'b0, 1'b1); // instrucao 2, endereco 02
            15: conferir(8'h05, 8'h20, 8'h82, 8'h00, 1'b0, 1'b1); // instrucao 3, endereco 04
            22: conferir(8'h07, 8'h10, 8'h81, 8'h05, 1'b0, 1'b0); // instrucao 4, endereco 06
            28: conferir(8'h09, 8'h20, 8'h83, 8'h05, 1'b0, 1'b0); // instrucao 5, endereco 08
            35: conferir(8'h0B, 8'hA0, 8'h83, 8'h05, 1'b0, 1'b0); // instrucao 6, endereco 0A
            41: conferir(8'h0D, 8'h20, 8'h1A, 8'h05, 1'b0, 1'b0); // instrucao 7, endereco 0C
            48: conferir(8'h0F, 8'h30, 8'h82, 8'h00, 1'b0, 1'b1); // instrucao 8, endereco 0E
            55: conferir(8'h11, 8'h10, 8'h80, 8'h07, 1'b0, 1'b0); // instrucao 9, endereco 10
            61: conferir(8'h13, 8'h20, 8'h82, 8'h07, 1'b0, 1'b0); // instrucao 10, endereco 12
            68: conferir(8'h15, 8'h30, 8'h83, 8'h05, 1'b0, 1'b0); // instrucao 11, endereco 14
            75: conferir(8'h17, 8'h10, 8'h85, 8'h04, 1'b0, 1'b0); // instrucao 12, endereco 16
            81: conferir(8'h19, 8'h80, 8'h83, 8'h04, 1'b0, 1'b0); // instrucao 13, endereco 18
            87: conferir(8'h09, 8'h20, 8'h08, 8'h04, 1'b0, 1'b0); // instrucao 14, endereco 08
            94: conferir(8'h0B, 8'hA0, 8'h83, 8'h04, 1'b0, 1'b0); // instrucao 15, endereco 0A
            100: conferir(8'h0D, 8'h20, 8'h1A, 8'h04, 1'b0, 1'b0); // instrucao 16, endereco 0C
            107: conferir(8'h0F, 8'h30, 8'h82, 8'h07, 1'b0, 1'b0); // instrucao 17, endereco 0E
            114: conferir(8'h11, 8'h10, 8'h80, 8'h0E, 1'b0, 1'b0); // instrucao 18, endereco 10
            120: conferir(8'h13, 8'h20, 8'h82, 8'h0E, 1'b0, 1'b0); // instrucao 19, endereco 12
            127: conferir(8'h15, 8'h30, 8'h83, 8'h04, 1'b0, 1'b0); // instrucao 20, endereco 14
            134: conferir(8'h17, 8'h10, 8'h85, 8'h03, 1'b0, 1'b0); // instrucao 21, endereco 16
            140: conferir(8'h19, 8'h80, 8'h83, 8'h03, 1'b0, 1'b0); // instrucao 22, endereco 18
            146: conferir(8'h09, 8'h20, 8'h08, 8'h03, 1'b0, 1'b0); // instrucao 23, endereco 08
            153: conferir(8'h0B, 8'hA0, 8'h83, 8'h03, 1'b0, 1'b0); // instrucao 24, endereco 0A
            159: conferir(8'h0D, 8'h20, 8'h1A, 8'h03, 1'b0, 1'b0); // instrucao 25, endereco 0C
            166: conferir(8'h0F, 8'h30, 8'h82, 8'h0E, 1'b0, 1'b0); // instrucao 26, endereco 0E
            173: conferir(8'h11, 8'h10, 8'h80, 8'h15, 1'b0, 1'b0); // instrucao 27, endereco 10
            179: conferir(8'h13, 8'h20, 8'h82, 8'h15, 1'b0, 1'b0); // instrucao 28, endereco 12
            186: conferir(8'h15, 8'h30, 8'h83, 8'h03, 1'b0, 1'b0); // instrucao 29, endereco 14
            193: conferir(8'h17, 8'h10, 8'h85, 8'h02, 1'b0, 1'b0); // instrucao 30, endereco 16
            199: conferir(8'h19, 8'h80, 8'h83, 8'h02, 1'b0, 1'b0); // instrucao 31, endereco 18
            205: conferir(8'h09, 8'h20, 8'h08, 8'h02, 1'b0, 1'b0); // instrucao 32, endereco 08
            212: conferir(8'h0B, 8'hA0, 8'h83, 8'h02, 1'b0, 1'b0); // instrucao 33, endereco 0A
            218: conferir(8'h0D, 8'h20, 8'h1A, 8'h02, 1'b0, 1'b0); // instrucao 34, endereco 0C
            225: conferir(8'h0F, 8'h30, 8'h82, 8'h15, 1'b0, 1'b0); // instrucao 35, endereco 0E
            232: conferir(8'h11, 8'h10, 8'h80, 8'h1C, 1'b0, 1'b0); // instrucao 36, endereco 10
            238: conferir(8'h13, 8'h20, 8'h82, 8'h1C, 1'b0, 1'b0); // instrucao 37, endereco 12
            245: conferir(8'h15, 8'h30, 8'h83, 8'h02, 1'b0, 1'b0); // instrucao 38, endereco 14
            252: conferir(8'h17, 8'h10, 8'h85, 8'h01, 1'b0, 1'b0); // instrucao 39, endereco 16
            258: conferir(8'h19, 8'h80, 8'h83, 8'h01, 1'b0, 1'b0); // instrucao 40, endereco 18
            264: conferir(8'h09, 8'h20, 8'h08, 8'h01, 1'b0, 1'b0); // instrucao 41, endereco 08
            271: conferir(8'h0B, 8'hA0, 8'h83, 8'h01, 1'b0, 1'b0); // instrucao 42, endereco 0A
            277: conferir(8'h0D, 8'h20, 8'h1A, 8'h01, 1'b0, 1'b0); // instrucao 43, endereco 0C
            284: conferir(8'h0F, 8'h30, 8'h82, 8'h1C, 1'b0, 1'b0); // instrucao 44, endereco 0E
            291: conferir(8'h11, 8'h10, 8'h80, 8'h23, 1'b0, 1'b0); // instrucao 45, endereco 10
            297: conferir(8'h13, 8'h20, 8'h82, 8'h23, 1'b0, 1'b0); // instrucao 46, endereco 12
            304: conferir(8'h15, 8'h30, 8'h83, 8'h01, 1'b0, 1'b0); // instrucao 47, endereco 14
            311: conferir(8'h17, 8'h10, 8'h85, 8'h00, 1'b0, 1'b1); // instrucao 48, endereco 16
            317: conferir(8'h19, 8'h80, 8'h83, 8'h00, 1'b0, 1'b1); // instrucao 49, endereco 18
            323: conferir(8'h09, 8'h20, 8'h08, 8'h00, 1'b0, 1'b1); // instrucao 50, endereco 08
            330: conferir(8'h0B, 8'hA0, 8'h83, 8'h00, 1'b0, 1'b1); // instrucao 51, endereco 0A
            336: conferir(8'h1B, 8'h20, 8'h1A, 8'h00, 1'b0, 1'b1); // instrucao 52, endereco 1A
            343: conferir(8'h1D, 8'hF0, 8'h82, 8'h23, 1'b0, 1'b0); // instrucao 53, endereco 1C
            344,345,346,347: begin
                if ({parado, erro, ac, pc, ri, rem, n, z} !==
                    {2'b10, 8'h23, 8'h1D, 8'hF0, 8'h82, 2'b00}) begin
                    $display("DIVERGENCIA no HLT ciclo=%0d parado=%b erro=%b", ciclos, parado, erro);
```

```
                    $finish;
                end
            end
            default: ;
        endcase
    end
    $display("SUCESSO: 53 pontos de verificacao, HLT no ciclo 344 e parada estavel.");
    $finish;
  end
endmodule
```

3.6 Testbench final das matrizes com atraso

Este testbench observa as portas do processador e confere 28 pontos da execução. Os resultados das subtrações são verificados antes das instruções STA. HLT é conferido no ciclo 183 e nos três ciclos seguintes. As nove transações de escrita em C foram também verificadas no teste comportamental da seção 3.1.

```
`timescale 1ns / 1ps

module neander_matrizes_timing_tb;

    reg clk;
    reg reset;

    wire parado, erro;
    wire [3:0] estado, opcode;
    wire [7:0] ac, pc, ri, rem;
    wire n, z;
    wire [7:0] endereco_memoria;
    wire [7:0] dado_memoria;
    wire [7:0] resultado_ula;

    integer ciclos;

    neander dut (
        .clk(clk),
        .reset(reset),
        .parado(parado),
        .erro(erro),
        .estado(estado),
        .ac(ac),
        .pc(pc),
        .ri(ri),
        .rem(rem),
        .opcode(opcode),
        .n(n),
        .z(z),
        .endereco_memoria(endereco_memoria),
        .dado_memoria(dado_memoria),
        .resultado_ula(resultado_ula)
    );

    // Referencia de programa_matrizes.coe. Observa apenas portas do top.
    initial begin
        clk = 0;
        forever #10 clk = ~clk;
    end

    task conferir;
        input [7:0] pc_ref, ri_ref, rem_ref, ac_ref;
        input n_ref, z_ref;
        begin
            if ({estado, pc, ri, rem, ac, n, z, parado, erro} !==
                {4'd2, pc_ref, ri_ref, rem_ref, ac_ref, n_ref, z_ref, 2'b00}) begin
                $display("DIVERGENCIA ciclo=%0d PC_instrucao=%h", ciclos, pc_ref-8'd1);
                $display("ESPERADO estado=2 PC=%h RI=%h REM=%h AC=%h N=%b Z=%b parado=0 erro=0",
                         pc_ref, ri_ref, rem_ref, ac_ref, n_ref, z_ref);
                $display("OBSERVADO estado=%h PC=%h RI=%h REM=%h AC=%h N=%b Z=%b parado=%b erro=%b",
                         estado, pc, ri, rem, ac, n, z, parado, erro);
```

```
                $finish;
            end
        end
    endtask

    initial begin
        reset = 1;
        repeat (10) @(negedge clk);
        reset = 0;
        for (ciclos = 1; ciclos <= 186; ciclos = ciclos + 1) begin
            @(negedge clk);
            #8; // Amostra apos a acomodacao dos atrasos, antes da proxima subida.
            $display("TRACE c=%0d estado=%h PC=%h RI=%h AC=%h N=%b Z=%b REM=%h endereco=%h dado=%h",
                     ciclos, estado, pc, ri, ac, n, z, rem, endereco_memoria, dado_memoria);
            case (ciclos)
                2: conferir(8'h01, 8'h20, 8'h00, 8'h00, 1'b0, 1'b1);
                9: conferir(8'h03, 8'h70, 8'h80, 8'h0A, 1'b0, 1'b0);
                16: conferir(8'h05, 8'h10, 8'h90, 8'h08, 1'b0, 1'b0);
                22: conferir(8'h07, 8'h20, 8'hA0, 8'h08, 1'b0, 1'b0);
                29: conferir(8'h09, 8'h70, 8'h81, 8'h05, 1'b0, 1'b0);
                36: conferir(8'h0B, 8'h10, 8'h91, 8'hFE, 1'b1, 1'b0);
                42: conferir(8'h0D, 8'h20, 8'hA1, 8'hFE, 1'b1, 1'b0);
                49: conferir(8'h0F, 8'h70, 8'h82, 8'h08, 1'b0, 1'b0);
                56: conferir(8'h11, 8'h10, 8'h92, 8'h00, 1'b0, 1'b1);
                62: conferir(8'h13, 8'h20, 8'hA2, 8'h00, 1'b0, 1'b1);
                69: conferir(8'h15, 8'h70, 8'h83, 8'h03, 1'b0, 1'b0);
                76: conferir(8'h17, 8'h10, 8'h93, 8'hFE, 1'b1, 1'b0);
                82: conferir(8'h19, 8'h20, 8'hA3, 8'hFE, 1'b1, 1'b0);
                89: conferir(8'h1B, 8'h70, 8'h84, 8'h0C, 1'b0, 1'b0);
                96: conferir(8'h1D, 8'h10, 8'h94, 8'h08, 1'b0, 1'b0);
                102: conferir(8'h1F, 8'h20, 8'hA4, 8'h08, 1'b0, 1'b0);
                109: conferir(8'h21, 8'h70, 8'h85, 8'h07, 1'b0, 1'b0);
                116: conferir(8'h23, 8'h10, 8'h95, 8'h06, 1'b0, 1'b0);
                122: conferir(8'h25, 8'h20, 8'hA5, 8'h06, 1'b0, 1'b0);
                129: conferir(8'h27, 8'h70, 8'h86, 8'h09, 1'b0, 1'b0);
                136: conferir(8'h29, 8'h10, 8'h96, 8'h06, 1'b0, 1'b0);
                142: conferir(8'h2B, 8'h20, 8'hA6, 8'h06, 1'b0, 1'b0);
                149: conferir(8'h2D, 8'h70, 8'h87, 8'h04, 1'b0, 1'b0);
                156: conferir(8'h2F, 8'h10, 8'h97, 8'hFB, 1'b1, 1'b0);
                162: conferir(8'h31, 8'h20, 8'hA7, 8'hFB, 1'b1, 1'b0);
                169: conferir(8'h33, 8'h70, 8'h88, 8'h06, 1'b0, 1'b0);
                176: conferir(8'h35, 8'h10, 8'h98, 8'h00, 1'b0, 1'b1);
                182: conferir(8'h37, 8'hF0, 8'hA8, 8'h00, 1'b0, 1'b1);
                183,184,185,186: begin
                    if ({estado, parado, erro, ac, pc, ri, rem, n, z} !==
                        {4'd10, 2'b10, 8'h00, 8'h37, 8'hF0, 8'hA8, 2'b01}) begin
                        $display("ERRO no HLT ciclo=%0d", ciclos);
                        $finish;
                    end
                end
                default: ;
            endcase
        end
        $display("SUCESSO: 28 pontos de verificacao das instrucoes.");
        $display("Resultados SUB conferidos: 08 FE 00 FE 08 06 06 FB 00.");
        $display("HLT no ciclo 183 e parada estavel por mais tres ciclos.");
        $display("Tempo do programa a 50 MHz: 3660 ns.");
        $finish;
    end
endmodule
```

4) Explicação e descrição das aplicações em Assembly

As listagens abaixo correspondem aos bytes dos arquivos COE fornecidos e utilizam a instrução SUB implementada neste projeto. Os endereços e os bytes das listagens são hexadecimais; os valores das matrizes e os resultados descritos no texto são decimais. LDA carrega o conteúdo de uma posição de memória, SUB subtrai esse conteúdo de AC e STA armazena AC na posição indicada.

4.1 Subtração de duas matrizes de ordem 3

A aplicação calcula C[i][j] = A[i][j] − B[i][j] para as nove posições. Os elementos são armazenados por linhas em posições consecutivas. A ocupa 80h–88h, B ocupa 90h–98h e C ocupa A0h–A8h. As posições destinadas a C são inicializadas com 55h para facilitar a identificação das escritas.

A = [10 5 8] B = [2 7 8] C = [8 -2 0]

[3 12 7] [5 4 1] [-2 8 6]

[9 4 6] [3 9 6] [6 -5 0]

Para cada elemento, o processador executa três instruções: LDA lê A, SUB subtrai B e STA grava C. O código repete explicitamente essa sequência para as nove posições, sem utilizar um laço ou endereçamento indireto.

Listagem correspondente ao programa_matrizes.coe

```
End.  Bytes   Assembly     Comentario
00    20 80   LDA 80h      ; AC recebe A[0][0]
02    70 90   SUB 90h      ; AC recebe A - B
04    10 A0   STA A0h      ; grava C[0][0]
06    20 81   LDA 81h      ; AC recebe A[0][1]
08    70 91   SUB 91h      ; AC recebe A - B
0A    10 A1   STA A1h      ; grava C[0][1]
0C    20 82   LDA 82h      ; AC recebe A[0][2]
0E    70 92   SUB 92h      ; AC recebe A - B
10    10 A2   STA A2h      ; grava C[0][2]
12    20 83   LDA 83h      ; AC recebe A[1][0]
14    70 93   SUB 93h      ; AC recebe A - B
16    10 A3   STA A3h      ; grava C[1][0]
18    20 84   LDA 84h      ; AC recebe A[1][1]
1A    70 94   SUB 94h      ; AC recebe A - B
1C    10 A4   STA A4h      ; grava C[1][1]
1E    20 85   LDA 85h      ; AC recebe A[1][2]
20    70 95   SUB 95h      ; AC recebe A - B
22    10 A5   STA A5h      ; grava C[1][2]
24    20 86   LDA 86h      ; AC recebe A[2][0]
26    70 96   SUB 96h      ; AC recebe A - B
28    10 A6   STA A6h      ; grava C[2][0]
2A    20 87   LDA 87h      ; AC recebe A[2][1]
2C    70 97   SUB 97h      ; AC recebe A - B
2E    10 A7   STA A7h      ; grava C[2][1]
30    20 88   LDA 88h      ; AC recebe A[2][2]
32    70 98   SUB 98h      ; AC recebe A - B
34    10 A8   STA A8h      ; grava C[2][2]
36    F0      HLT          ; encerra o programa
```

Na primeira sequência, LDA 80h carrega 10, SUB 90h subtrai 2 e STA A0h grava 8. Na segunda, 5 − 7 produz −2, representado por FEh. Em oito bits, a subtração equivale a somar o complemento de dois: 05h + (~07h + 1) = 05h + F9h = FEh, descartando qualquer transporte além do oitavo bit. O resultado FEh corresponde a −2 na interpretação com sinal.

O indicador N recebe o bit 7 do resultado e Z é ativado quando o resultado é zero. Assim, −2 e −5 produzem N = 1 e Z = 0; as subtrações 8 − 8 e 6 − 6 produzem N = 0 e Z = 1. Todos os resultados deste exemplo estão no intervalo representável em complemento de dois de 8 bits, de −128 a 127.

A sequência esperada em A0h–A8h é 08h, FEh, 00h, FEh, 08h, 06h, 06h, FBh e 00h. São executadas 28 instruções: nove LDA, nove SUB, nove STA e um HLT. Na implementação da unidade de controle, LDA e SUB consomem sete ciclos cada, STA consome seis e HLT consome três, totalizando 9 × (7 + 7 + 6) + 3 = 183 ciclos. A 50 MHz, o período é 20 ns e o tempo do programa é 3,66 µs, sem incluir reset e observações adicionais do testbench.

4.2 Multiplicação por somas sucessivas

A aplicação calcula 7 × 5 por cinco adições de 7. O resultado é inicializado com zero e o contador recebe o multiplicador. No início de cada repetição, o programa testa se o contador é zero. Caso seja, carrega o resultado e executa HLT. Caso contrário, soma 7 ao resultado, decrementa o contador por meio de SUB e retorna ao teste.

```
Endereco  Conteudo inicial  Funcao
80h       07h               Multiplicando A = 7
81h       05h               Multiplicador B = 5
82h       00h               Resultado acumulado
83h       00h               Contador de repeticoes
84h       00h               Constante zero
85h       01h               Constante um
```

Listagem correspondente ao programa_multiplicacao.coe

```
End. Bytes   Assembly       Comentario
00   20 84   LDA 84h        ; carrega zero
02   10 82   STA 82h        ; resultado recebe zero
04   20 81   LDA 81h        ; carrega B = 5
06   10 83   STA 83h        ; contador recebe B
08   20 83   LACO: LDA 83h  ; carrega o contador e atualiza Z
0A   A0 1A   JZ 1Ah         ; vai ao fim se contador = 0
0C   20 82   LDA 82h        ; carrega resultado parcial
0E   30 80   ADD 80h        ; soma A = 7
10   10 82   STA 82h        ; grava novo resultado
12   20 83   LDA 83h        ; carrega contador
14   70 85   SUB 85h        ; decrementa contador
16   10 83   STA 83h        ; grava contador
18   80 08   JMP 08h        ; volta ao inicio do laco
1A   20 82   FIM: LDA 82h   ; carrega resultado final
1C   F0      HLT            ; encerra o programa
```

Os rótulos LACO e FIM são explicativos e correspondem a 08h e 1Ah. Após cada repetição, os pares resultado/contador são 7/4, 14/3, 21/2, 28/1 e 35/0. A última passagem por LDA 83h estabelece Z = 1 e permite que JZ alcance FIM. Em seguida, LDA 82h carrega 35 e atualiza as flags para N = 0 e Z = 0. Portanto, Z = 0 ao final é esperado, embora o contador tenha chegado a zero.

O programa contém 15 instruções estáticas, mas executa 53 instruções devido às repetições: quatro de inicialização, cinco passagens de nove instruções, duas para o teste final do contador e duas para carregar o resultado e parar. A execução consome 344 ciclos, equivalentes a 6,88 µs a 50 MHz. A saída final é AC = 23h (35), PC = 1Dh, RI = F0h e REM = 82h. O exemplo usa operandos positivos e produto representável em oito bits.

5) Uso do Hidra e geração do binário para inicializar a memória

5.1 Adaptação da multiplicação ao Neander do Hidra

A aplicação de multiplicação foi adaptada para utilizar apenas o conjunto de instruções do Neander selecionado no Hidra. O decremento por SUB com a constante 1 foi substituído por ADD com a constante FFh, que representa −1 em complemento de dois de 8 bits. Por exemplo, 05h + FFh = 104h; mantendo os oito bits menos significativos, obtém-se 04h. Essa adaptação preserva os endereços e o número de instruções, pois ADD e SUB ocupam dois bytes no processador implementado.

No endereço 14h, o byte 70h da versão com SUB foi substituído por 30h, correspondente a ADD. O operando continua sendo 85h. Na posição 85h, a constante 01h foi substituída por FFh. Os demais bytes da imagem inicial permanecem iguais.

```
ORG 0
    LDA ZERO
    STA RESULTADO
    LDA B
    STA CONTADOR
LACO:
    LDA CONTADOR
    JZ FIM
    LDA RESULTADO
    ADD A
    STA RESULTADO
    LDA CONTADOR
    ADD MENOS_UM
    STA CONTADOR
    JMP LACO
FIM:
    LDA RESULTADO
    HLT
ORG 128
A:          DB 7
B:          DB 5
RESULTADO:  DB 0
CONTADOR:   DB 0
ZERO:       DB 0
MENOS_UM:   DB 255
```

5.2 Montagem e execução no Hidra

O programa foi montado e executado no Hidra com a arquitetura Neander. Ao final, AC e a posição 82h continham 23h (35), a posição 83h continha 00h, PC era 1Dh e as flags N e Z estavam em zero. O simulador informou 53 instruções e 141 acessos.

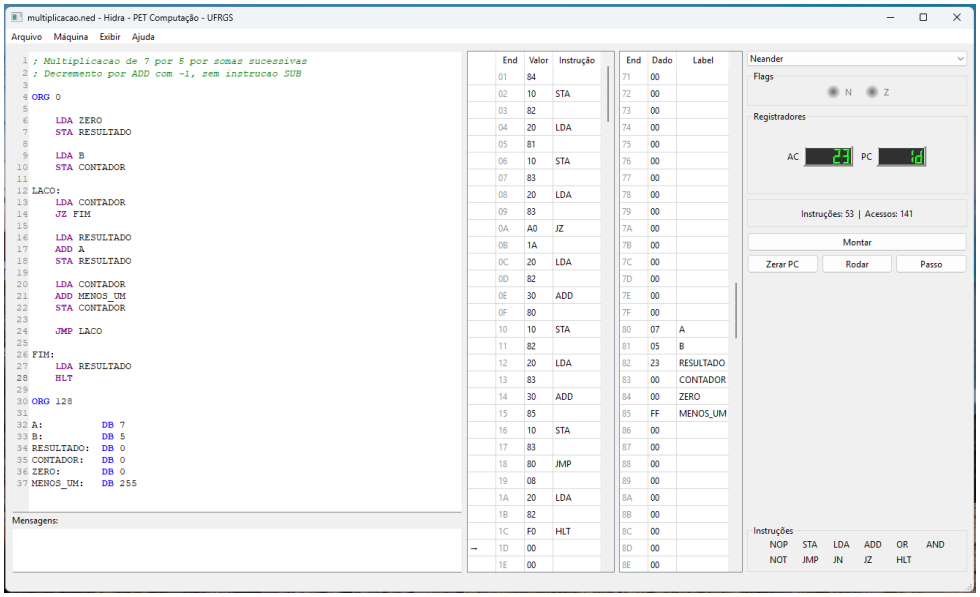


Figura 1 — Execução no Hidra da multiplicação sem SUB, com resultado 23h e contador zero.

5.3 Exportação da memória e conversão para COE

Após uma nova montagem, antes da execução, foi utilizada a opção Máquina → Exportar memória. O arquivo multiplicacao_hidra.mem possui 516 bytes: um cabeçalho de quatro bytes (03h, 4Eh, 44h, 52h), seguido por 256 pares de bytes. Neste arquivo, todos os segundos bytes dos pares são zero. A conversão removeu o cabeçalho e extraiu o primeiro byte de cada par, preservando a ordem das 256 posições de memória. O vetor foi escrito em hexadecimal no arquivo multiplicacao_hidra.coe, com as diretivas memory_initialization_radix=16 e memory_initialization_vector.

A conferência verificou todas as 256 palavras convertidas. Os dados iniciais em 80h–85h são 07h, 05h, 00h, 00h, 00h e FFh. Assim, a imagem exportada contém resultado e contador zerados, e não os valores alterados pela execução anterior.

5.4 Execução da imagem do Hidra na BRAM do processador

O COE convertido foi selecionado na memória do projeto Vivado e executado em simulação comportamental com neander_multiplicacao_tb. O teste confirmou as somas parciais 7, 14, 21, 28 e 35, o contador de 5 até 0, 12 escritas, 53 instruções e 344 ciclos. A 50 MHz, o tempo de execução é 6880 ns, ou 6,88 µs. O encerramento do testbench em 6960 ns inclui reset e observação posterior ao HLT.

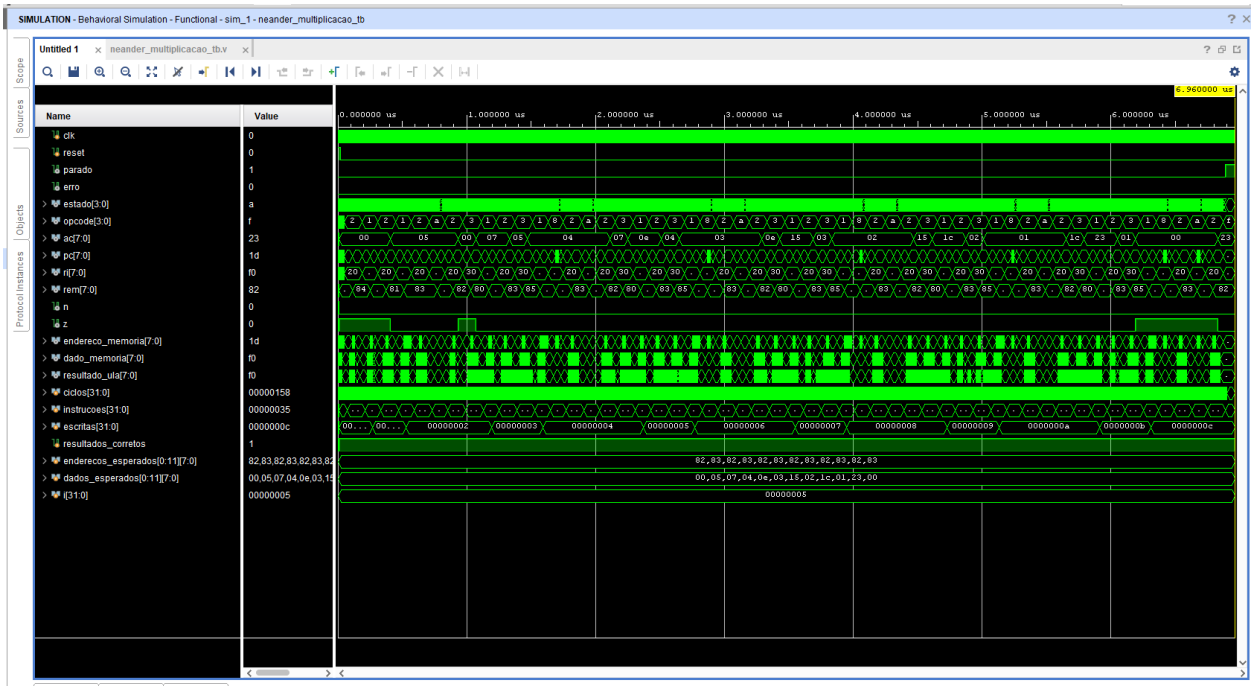


Figura 2 — Simulação comportamental no Vivado com a imagem de memória montada no Hidra e convertida para COE.

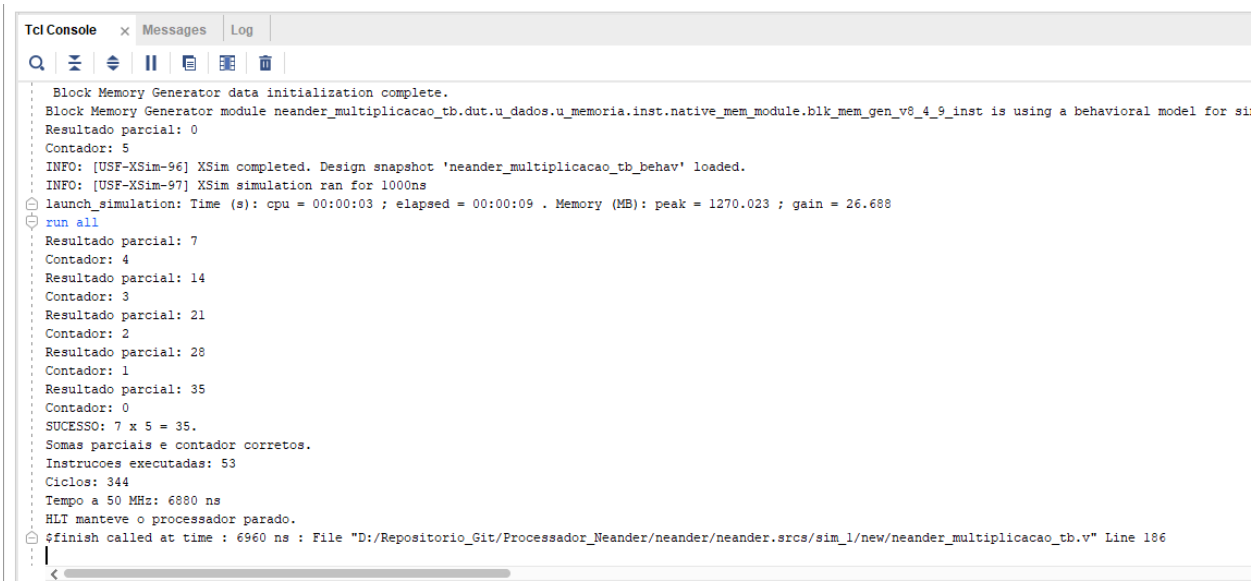


Figura 3 — Console confirma o resultado e as verificações do testbench com o programa sem SUB.

A imagem da multiplicação exportada pelo Hidra foi executada na BRAM do processador Verilog e validada até a simulação temporal descrita na seção 6.4. A aplicação de matrizes validada no Vivado utiliza SUB nativa. A versão equivalente das matrizes para o Hidra é apresentada nas seções 5.5 e 5.6.

5.5 Subtração das matrizes no Assembly do Hidra

Para executar a subtração no conjunto de instruções do Hidra, cada elemento de B é complementado com NOT e incrementado com ADD UM. O valor obtido é armazenado em TEMP e somado ao elemento de A. A aritmética de oito bits produz C = A − B em complemento de dois. As matrizes A, B e C iniciam em 80h, 90h e A0h. UM e TEMP ocupam B0h e B1h.

```
ORG 0
    LDA B0
```

```
    NOT
    ADD UM
    STA TEMP
    LDA A0
    ADD TEMP
    STA C0
    LDA B1
    NOT
    ADD UM
    STA TEMP
    LDA A1
    ADD TEMP
    STA C1
    LDA B2
    NOT
    ADD UM
    STA TEMP
    LDA A2
    ADD TEMP
    STA C2
    LDA B3
    NOT
    ADD UM
    STA TEMP
    LDA A3
    ADD TEMP
    STA C3
    LDA B4
    NOT
    ADD UM
    STA TEMP
    LDA A4
    ADD TEMP
    STA C4
    LDA B5
    NOT
    ADD UM
    STA TEMP
    LDA A5
    ADD TEMP
    STA C5
    LDA B6
    NOT
    ADD UM
    STA TEMP
    LDA A6
    ADD TEMP
    STA C6
    LDA B7
    NOT
    ADD UM
    STA TEMP
    LDA A7
    ADD TEMP
    STA C7
    LDA B8
    NOT
    ADD UM
    STA TEMP
    LDA A8
    ADD TEMP
    STA C8
    HLT
ORG 128
A0: DB 10
A1: DB 5
A2: DB 8
A3: DB 3
A4: DB 12
A5: DB 7
A6: DB 9
A7: DB 4
```

```
A8: DB 6
ORG 144
B0: DB 2
B1: DB 7
B2: DB 8
B3: DB 5
B4: DB 4
B5: DB 1
B6: DB 3
B7: DB 9
B8: DB 6
ORG 160
C0: DB 0
C1: DB 0
C2: DB 0
C3: DB 0
C4: DB 0
C5: DB 0
C6: DB 0
C7: DB 0
C8: DB 0
ORG 176
UM:   DB 1
TEMP: DB 0
```

A execução no Hidra confirmou os nove valores 08h, FEh, 00h, FEh, 08h, 06h, 06h, FBh e 00h nas posições A0h a A8h. O simulador registrou 64 instruções e 172 acessos à memória. Ao final, AC = 00h, PC = 76h, N = 0 e Z = 1. A posição temporária B1h continha FAh, o complemento de dois do último elemento de B, igual a 6. Os resultados coincidem com a conferência independente do Assembly e com a matriz obtida pela aplicação com SUB nativa no Vivado. A imagem inicial foi exportada e conferida, conforme a seção 5.6.

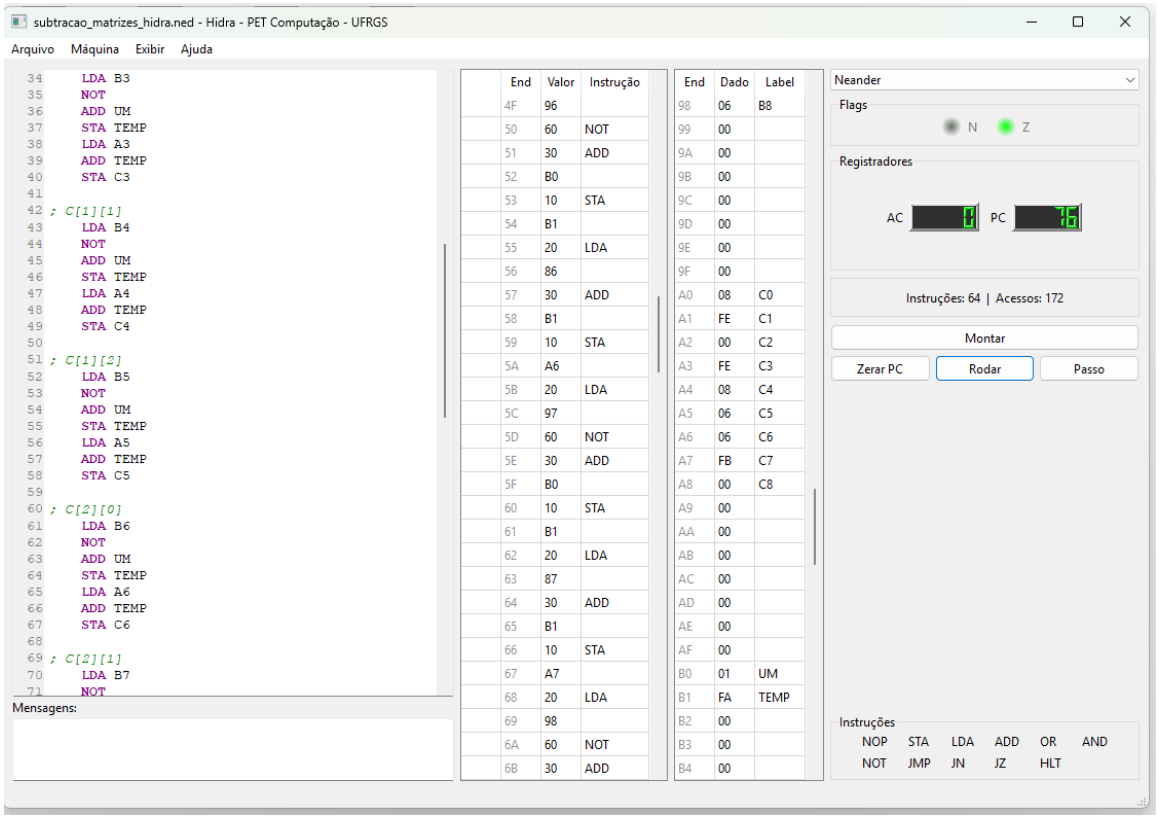


Figura 4 — Subtração de matrizes executada no Hidra por complemento de dois. A memória A0h a A8h apresenta os nove resultados e o painel registra 64 instruções e 172 acessos.

5.6 Memória das matrizes exportada pelo Hidra

O arquivo subtracao_matrizes_hidra.mem foi exportado após uma nova montagem e antes da execução. Seus 516 bytes contêm o cabeçalho 03h, 4Eh, 44h, 52h e 256 palavras de 16 bits em ordem de byte menos significativo primeiro. Os bytes superiores das palavras são zero. A conversão para subtracao_matrizes_hidra.coe preservou o byte inferior de cada posição e produziu 256 entradas de oito bits para a inicialização da BRAM.

As 256 posições foram comparadas com uma montagem independente do Assembly da seção 5.5 e coincidiram. A ocupa 80h a 88h, B ocupa 90h a 98h e C ocupa A0h a A8h. C e TEMP foram exportados com zero, enquanto UM contém 01h em B0h. A execução independente dos bytes exportados confirmou os nove resultados, 64 instruções e PC final 76h. Essa conferência verifica a conversão da memória. Os ensaios das matrizes no Vivado, apresentados nas seções 6 e 7, utilizam programa_matrizes.coe, com SUB nativa.

```
memory_initialization_radix=16;
memory_initialization_vector=
20,90,60,30,B0,10,B1,20,
80,30,B1,10,A0,20,91,60,
30,B0,10,B1,20,81,30,B1,
10,A1,20,92,60,30,B0,10,
B1,20,82,30,B1,10,A2,20,
93,60,30,B0,10,B1,20,83,
30,B1,10,A3,20,94,60,30,
B0,10,B1,20,84,30,B1,10,
A4,20,95,60,30,B0,10,B1,
20,85,30,B1,10,A5,20,96,
60,30,B0,10,B1,20,86,30,
B1,10,A6,20,97,60,30,B0,
10,B1,20,87,30,B1,10,A7,
20,98,60,30,B0,10,B1,20,
88,30,B1,10,A8,F0,00,00,
00,00,00,00,00,00,00,00,
0A,05,08,03,0C,07,09,04,
06,00,00,00,00,00,00,00,
02,07,08,05,04,01,03,09,
06,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
01,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00,
00,00,00,00,00,00,00,00;
```

6) Simulações sem e com atraso

6.1 Subtração de matrizes em simulação comportamental

A simulação comportamental verificou nove transações de escrita, com resultados 8, −2, 0, −2, 8, 6, 6, −5 e 0. Ao final, AC = 00h, PC = 37h, RI = F0h, REM = A8h, N = 0, Z = 1, parado = 1 e erro = 0. A captura geral foi obtida antes da revisão final de op_ula e Z. O teste foi repetido após essa revisão e confirmou os mesmos nove resultados, 28 instruções e 183 ciclos.

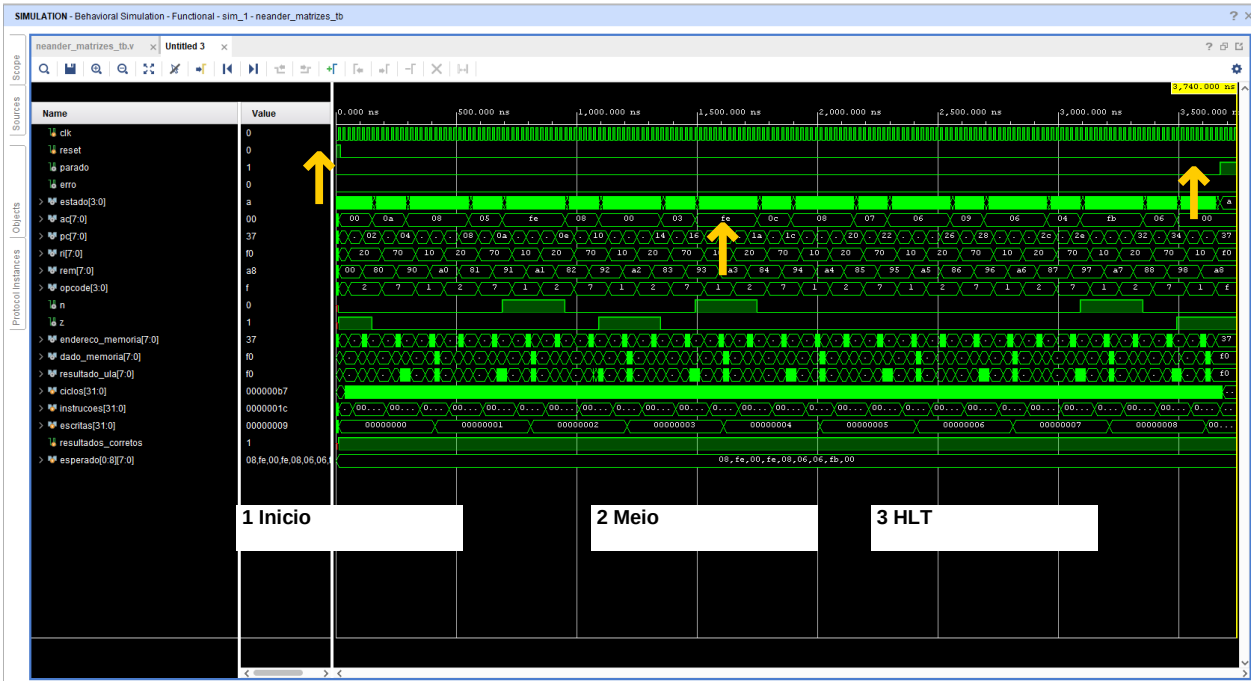


Figura 5 — Visão geral da simulação comportamental da subtração de matrizes. As flechas indicam início, trecho intermediário e parada.

As flechas indicam o reset e a primeira busca, a execução intermediária das sequências LDA, SUB e STA e a parada após a nona escrita. Nesta escala, os barramentos ficam comprimidos. A console abaixo apresenta os valores individuais e as contagens verificadas.

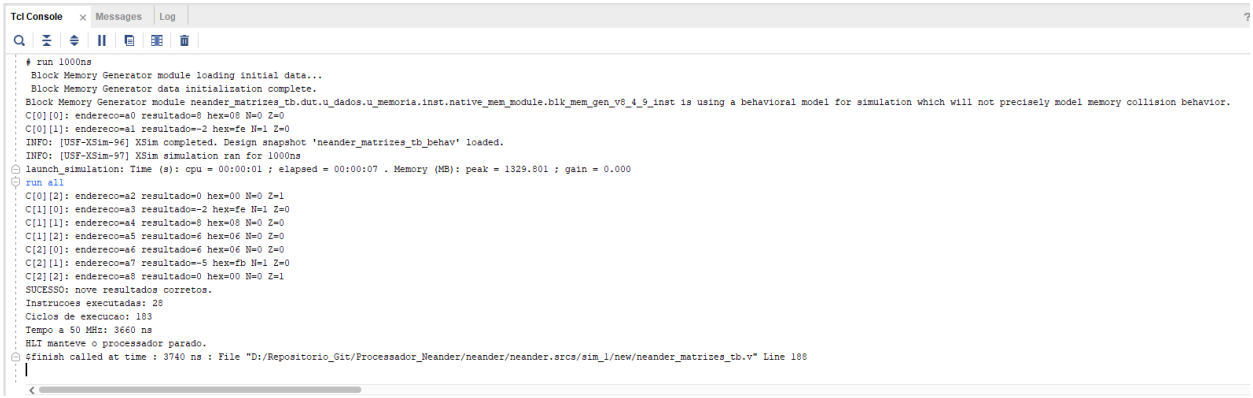


Figura 6 — Console com os nove resultados, 28 instruções e 183 ciclos. O encerramento em 3740 ns inclui reset e observação posterior à execução.

6.2 Multiplicação em simulação comportamental

Após a atualização da unidade de controle, a multiplicação foi novamente aprovada. As somas parciais foram 7, 14, 21, 28 e 35, enquanto o contador foi reduzido de 5 até 0. Foram conferidas 12 escritas, 53 instruções e 344 ciclos.

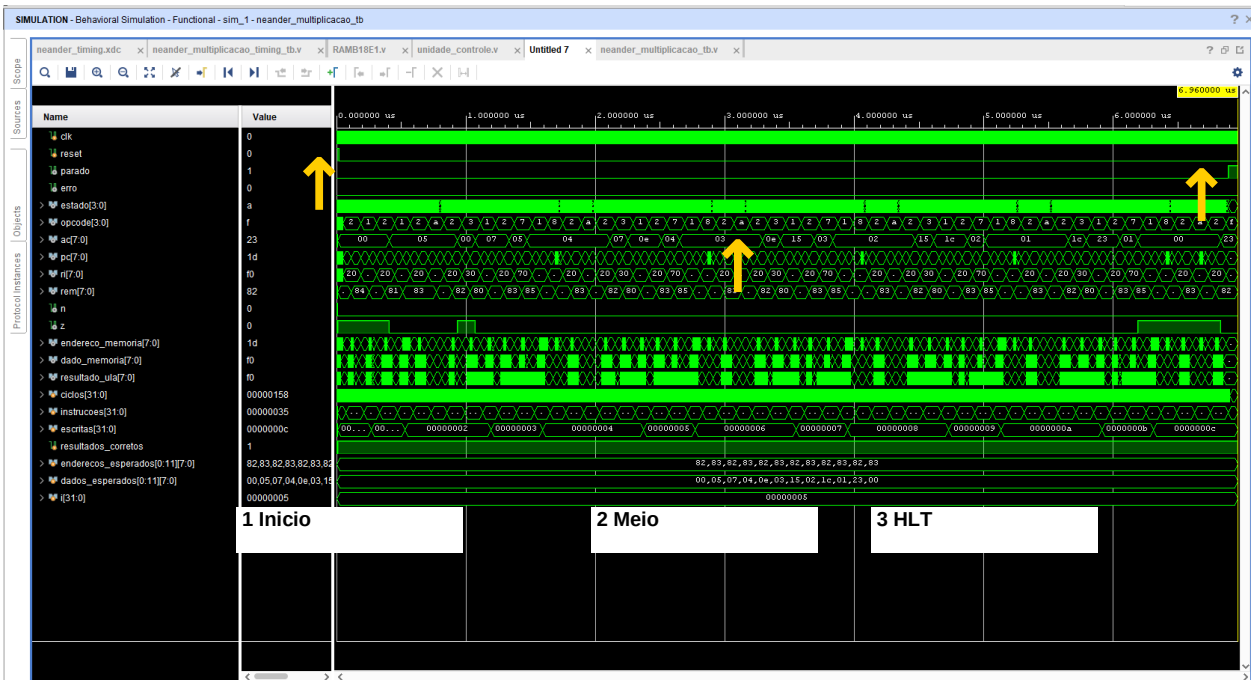


Figura 7 — Simulação comportamental da multiplicação com a unidade de controle atualizada. As flechas indicam início, trecho intermediário e parada.

As flechas indicam a inicialização do resultado em zero e do contador em 5, a atualização do resultado e do contador durante o laço e a parada com AC = 23h, PC = 1Dh e parado = 1. As ampliações da simulação temporal são apresentadas na seção 6.4.

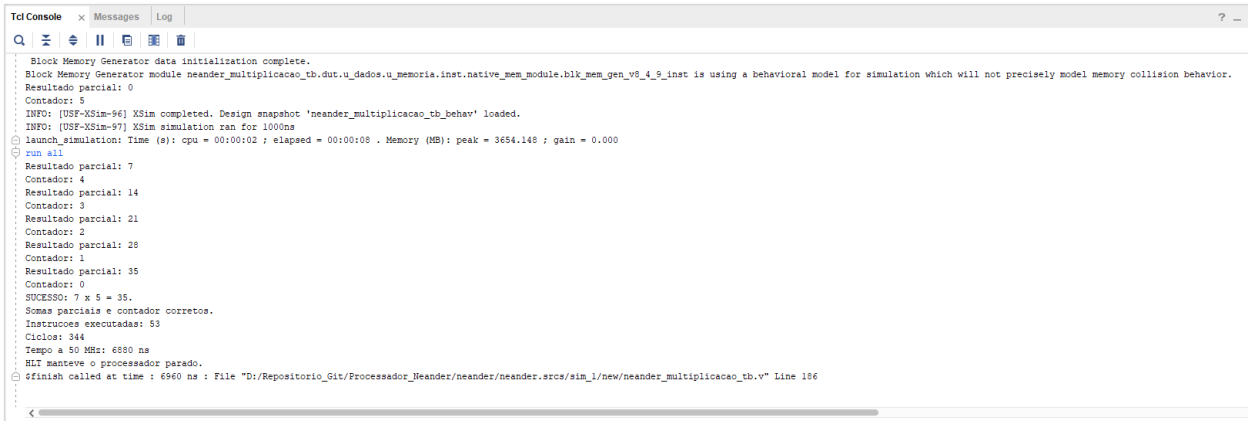


Figura 8 — Console confirma 7 × 5 = 35, somas parciais, 344 ciclos e estabilidade após HLT.

6.3 Simulação funcional após implementação

A netlist implementada foi testada com launch_simulation -mode post-implementation -type functional. A console informou sucesso, resultado 35, 344 ciclos e manutenção dos valores finais após HLT. O teste terminou em 7141 ns. Esse instante inclui 200 ns de reset e observações adicionais, enquanto o tempo do programa em ciclos é 6880 ns.

6.4 Simulação temporal da multiplicação após implementação

Na revisão anterior, o primeiro ponto de controle que divergiu foi o ciclo 9: após LDA ZERO, AC era 00h e a flag Z observada era 0. Na revisão testada, Z é obtida pela comparação do valor registrado de AC com 00h. Essa relação preserva o comportamento da unidade de controle, que habilita a atualização de AC e das flags nas mesmas instruções. A atribuição está reproduzida na seção 2.3.

Com a memória multiplicacao_hidra.coe e clock de 50 MHz, o XSim confirmou a anotação SDF no DUT da netlist implementada. A execução utilizou o canto lento e os atrasos máximos. No ciclo 9, AC = 00h e Z = 1. No ciclo 341, AC recebeu 23h (35), com N = 0 e Z = 0. O teste atingiu HLT no ciclo 344 com PC = 1Dh e RI = F0h. Os valores permaneceram estáveis nos três ciclos seguintes. Os 53 pontos de verificação passaram. O tempo de execução do programa foi 6,88 µs; o testbench encerrou em 7,148 µs, após reset e verificação da parada.

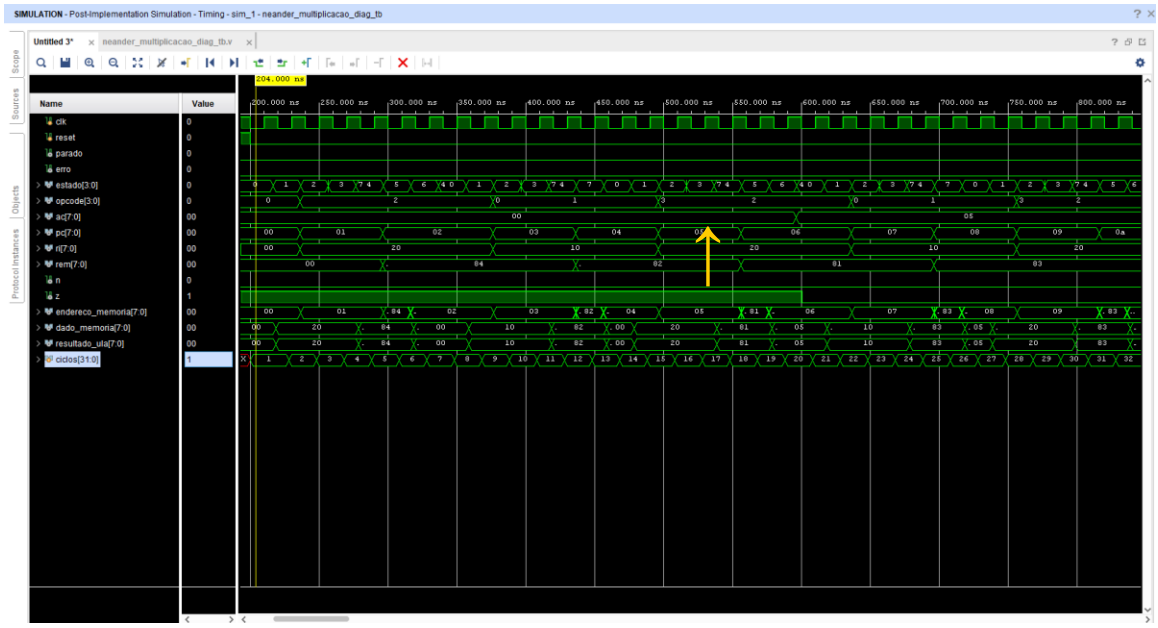


Figura 9 — Início da multiplicação na simulação temporal, de 200 a 800 ns.

O programa inicializa RESULTADO em 82h com 00h e CONTADOR em 83h com 05h. As flags acompanham a leitura do valor 5, com N = 0 e Z = 0.

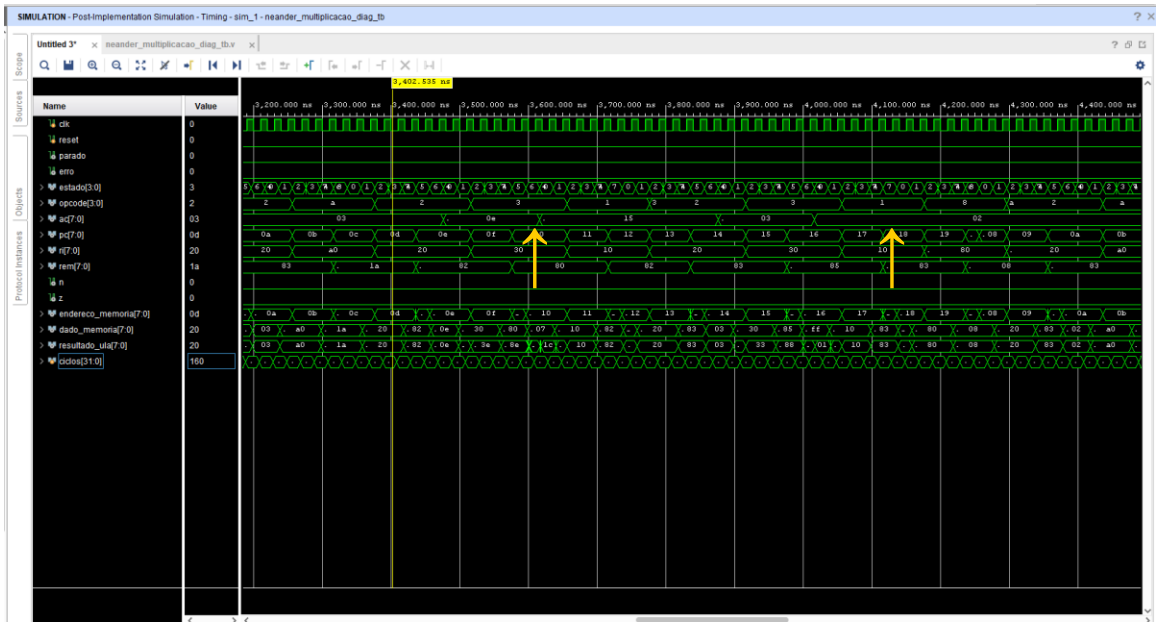


Figura 10 — Trecho intermediário da multiplicação.

Na terceira repetição, AC recebe 15h (21) após somar 0Eh (14) com 07h. O resultado é armazenado em 82h. Em seguida, ADD do valor FFh decrementa o contador de 03h para 02h, e STA grava esse valor em 83h.

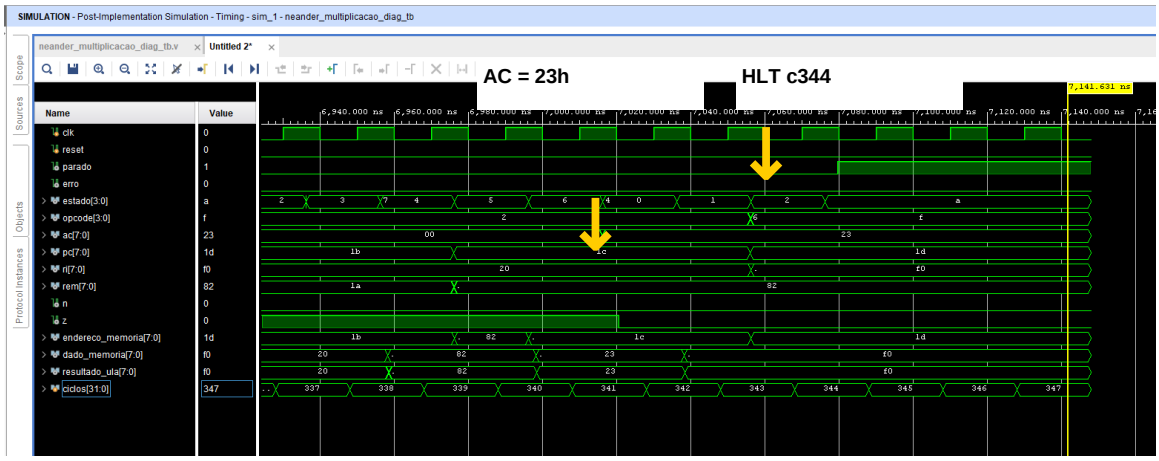


Figura 11 — Final da multiplicação na simulação temporal. AC recebe 23h no ciclo 341. O sinal parado sobe no ciclo 344 e permanece ativo até o ciclo 347.

Os dois programas foram aprovados nas simulações comportamentais e nas verificações temporais após implementação. As capturas ampliadas mostram o início, uma etapa intermediária e o final de cada aplicação. As flechas destacam a inicialização, os resultados das operações e a parada após HLT.

6.5 Simulação temporal da subtração de matrizes

Com programa_matrizes.coe, o testbench temporal aprovou 28 pontos de verificação e conferiu os resultados 08h, FEh, 00h, FEh, 08h, 06h, 06h, FBh e 00h. Em complemento de dois, FEh representa −2 e FBh representa −5. O processador atingiu HLT no ciclo 183 e permaneceu estável nos três ciclos seguintes.

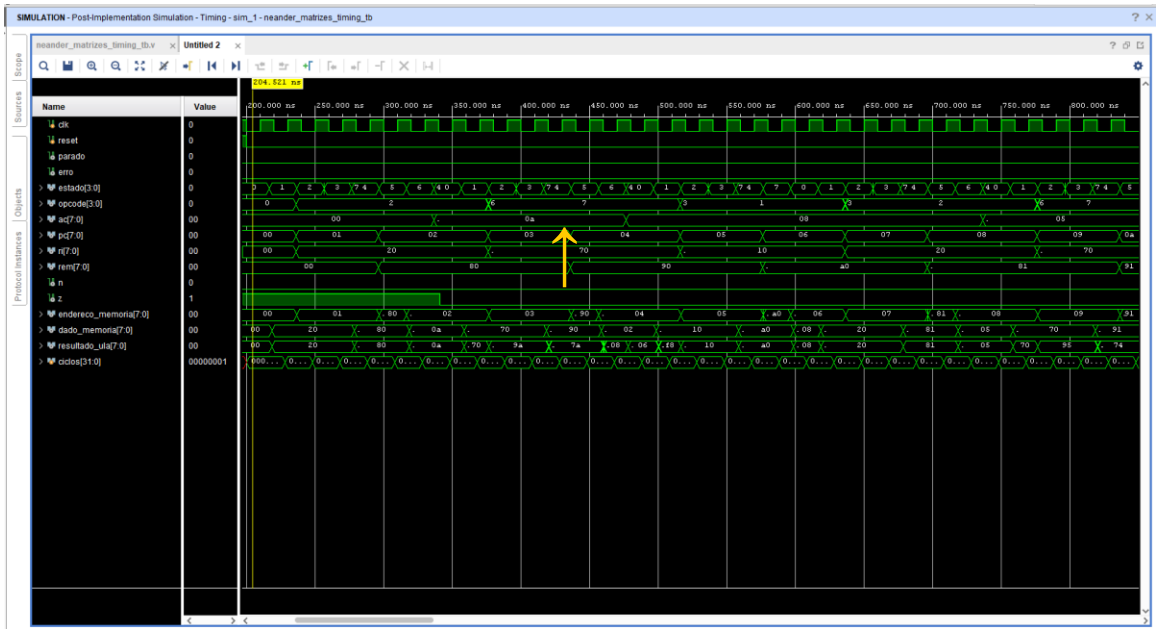


Figura 12 — Início da simulação temporal das matrizes, de 200 a 800 ns. LDA carrega 0Ah, SUB produz 08h e STA seleciona A0h para o armazenamento do primeiro resultado.

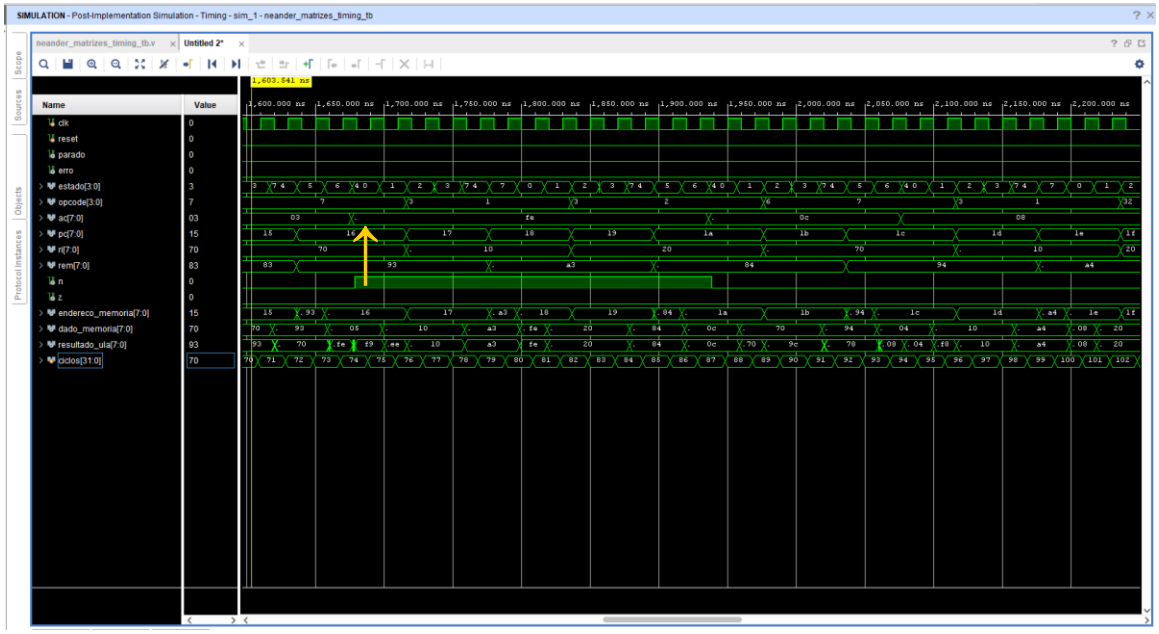


Figura 13 — Trecho intermediário, de 1600 a 2200 ns. A operação 03h − 05h produz FEh (−2), com N = 1 e Z = 0. Em seguida, 0Ch − 04h produz 08h e N volta a zero. Os endereços A3h e A4h são selecionados pelas respectivas instruções STA.

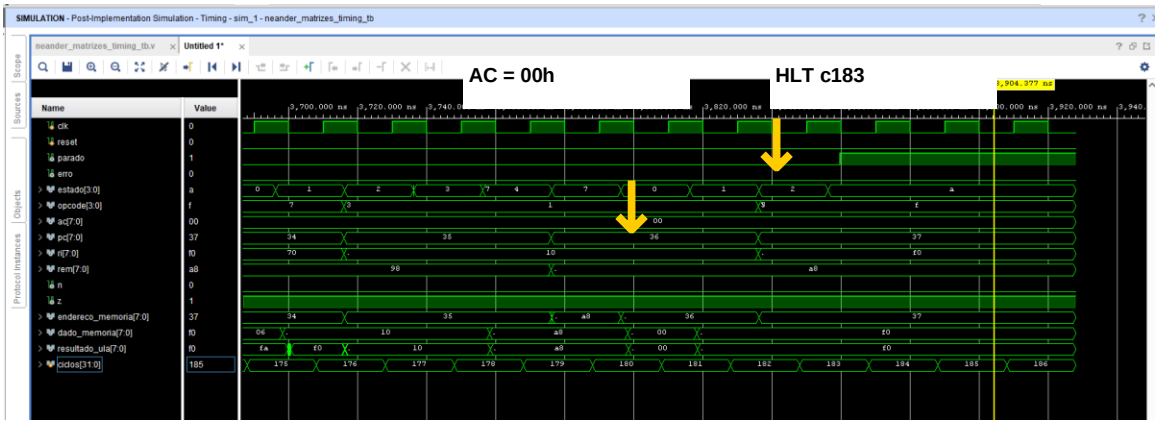


Figura 14 — Final da simulação temporal das matrizes. No ciclo 185, AC = 00h, PC = 37h, RI = F0h, REM = A8h, N = 0, Z = 1, parado = 1 e erro = 0.

A duração do programa foi 183 × 20 ns = 3660 ns, ou 3,66 µs. O encerramento do testbench em 3928 ns inclui o reset inicial e a observação após HLT.

7) Dados de área e tempo de execução

Programa	Número de Instruções Executadas	Tempo de execução em # de ciclos de relógio (c.c.)	Tempo de execução em Segundos (Neander operando a 50 MHz)
Subtração de matrizes	28	183	0,00000366 s
Multiplicação por somas sucessivas	53	344	0,00000688 s

Para f = 50 MHz, T = 1/f = 20 ns. O tempo de execução é o número de ciclos multiplicado por 20 ns. Os dois programas tiveram suas contagens confirmadas na simulação comportamental e na simulação temporal após implementação. A contagem de instruções inclui HLT. Reset e observações posteriores estão excluídos dos tempos da tabela.

Dados de área do Neander

FPGA device: AMD Artix-7 XC7A100T-CSG324-1.

Número de 4-LUTs — o Vivado reportou 71 Slice LUTs do Artix-7, todas utilizadas como lógica. Como as LUTs desse dispositivo têm até seis entradas, o valor foi mantido na unidade nativa do relatório. Não foi calculada uma equivalência em 4-LUTs.

Número de flip-flops — 44.

Número de BRAMs — uma RAMB18E1, equivalente a 0,5 Block RAM Tile.

Número de DSPs — zero.

O projeto utiliza ainda 25 slices. A multiplicação é realizada pelo programa de somas sucessivas executado na ULA.

A unidade de controle utiliza 8 LUTs e 12 flip-flops. O caminho de dados utiliza 63 LUTs e 32 flip-flops. O relatório também registra 70 IOBs e um buffer global de clock.

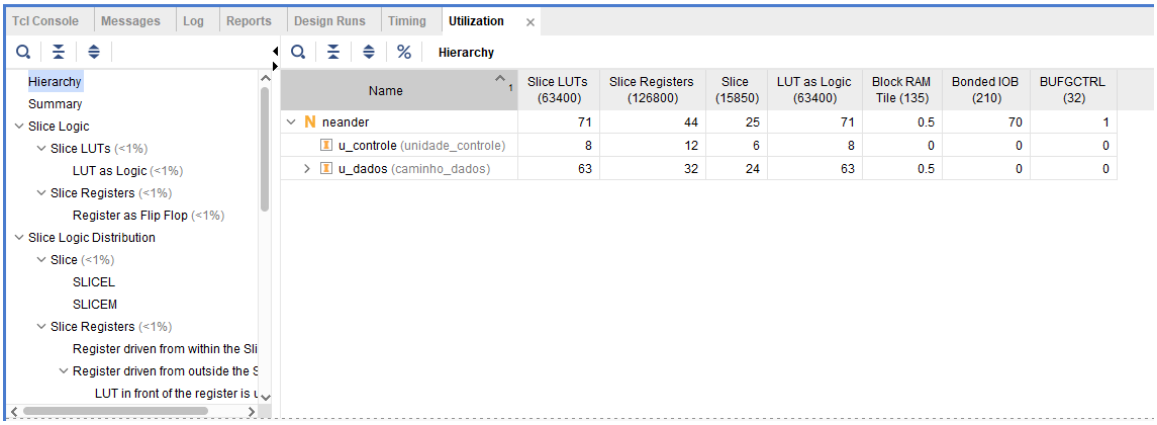


Figura 15 — Recursos utilizados pelo Neander após posicionamento e roteamento.

Para o clock de 50 MHz, a pior folga de setup foi +12,714 ns e a pior folga de hold foi +0,143 ns. A menor folga de largura de pulso foi +9,500 ns. Todos os pontos avaliados atenderam às restrições especificadas. O Check Timing registrou avisos de ausência de atrasos de entrada e saída. A conclusão de atendimento temporal se refere às restrições efetivamente especificadas no projeto.

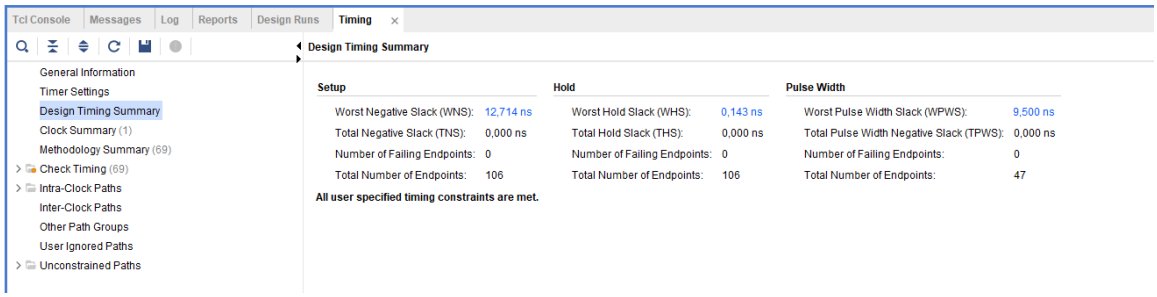


Figura 16 — Resumo da análise estática de temporização para o clock de 50 MHz.

Conclusão final

O processador Neander de 8 bits foi implementado em Verilog com unidade de controle, caminho de dados e memória BRAM. A instrução SUB foi incorporada à ULA e à decodificação, permitindo executar subtrações com operandos de memória e representar resultados negativos em complemento de dois.

Os testes confirmaram a subtração de duas matrizes 3 × 3 e a multiplicação de 7 por 5 por somas sucessivas. As matrizes produziram nove resultados corretos em 28 instruções e 183 ciclos. A multiplicação produziu 35 em 53 instruções e 344 ciclos. A 50 MHz, os tempos foram 3,66 µs e 6,88 µs, respectivamente. As verificações temporais após implementação também foram aprovadas, com estabilidade dos valores finais após HLT.

A execução dos dois programas no Hidra confirmou o resultado 23h da multiplicação e os nove elementos da subtração de matrizes. As imagens de memória iniciais foram exportadas e convertidas para COE. No programa das matrizes, NOT e ADD de uma unidade formam o complemento de dois do subtraendo. Na multiplicação, ADD de FFh decrementa o contador em oito bits.

A implementação utilizou 71 LUTs, 44 flip-flops, 25 slices e uma RAMB18E1 na FPGA Artix-7 XC7A100T. A análise estática apresentou folgas positivas de setup e hold para as restrições avaliadas. Os avisos de ausência de atrasos de entrada e saída limitam a cobertura das interfaces externas. Os resultados apresentados correspondem à síntese, à implementação e às simulações realizadas no Vivado.

A depuração da flag Z mostrou a importância de comparar a simulação comportamental com a netlist implementada. Na revisão validada, Z é derivada do valor registrado no acumulador. Os testes dos dois programas confirmaram a operação dessa revisão com clock de 50 MHz.